# Terraform — Interview Ready Answers

*Comprehensive answers with examples for all 36 Terraform questions*

36 Questions • State Management • Modules • Drift • Lifecycle • Providers • Policy-as-Code • Multi-Account • IaC Best Practices

---

## Q1. How do you handle Terraform state file corruption?

**Answer**

State file corruption is a critical incident. The state file is the source of truth about what Terraform manages — corruption means Terraform loses track of real infrastructure. A clear, practiced recovery process is essential.

**Prevention First — Why Corruption Happens**

- Concurrent applies — Two engineers or pipelines running terraform apply simultaneously on the same state (without locking).
- Interrupted apply — Network failure or process kill mid-write leaves a partial state.
- Manual edits — Directly editing the JSON state file incorrectly.
- Backend misconfiguration — Writing to wrong path or wrong storage account.

**Step 1 — Stop All Terraform Operations Immediately**

As soon as corruption is detected, prevent any further writes to the state. Alert the team and ensure no pipeline continues running terraform apply against this state.

**Step 2 — Inspect the Corrupted State**

```
terraform show                          # Check current state
cat terraform.tfstate | python -m json.tool   # Validate JSON
terraform state list                    # List known resources
```

If the state is valid JSON but logically inconsistent, you may be able to recover specific resources. If it is not valid JSON at all, proceed to backup restoration.

**Step 3 — Restore from Backup**

Azure Blob Storage and AWS S3 backends automatically version every state write. Retrieve the last known-good version:

```
# AWS S3 — list versions
aws s3api list-object-versions \
  --bucket my-tfstate-bucket --prefix prod/terraform.tfstate

# AWS S3 — restore specific version
aws s3api get-object \
  --bucket my-tfstate-bucket \
  --key prod/terraform.tfstate \
  --version-id <VERSION_ID> terraform.tfstate.restored

# Azure Blob Storage — restore previous version
az storage blob restore \
  --account-name tfstateaccount \
  --container-name tfstate \
  --blob-name prod/terraform.tfstate
```

**Step 4 — Validate After Restoration**

```
# Replace corrupted state with restored version
cp terraform.tfstate.restored terraform.tfstate

# Reinitialise and validate
terraform init
terraform plan    # Should show no changes if state matches reality
```

If terraform plan shows unexpected changes after restoration, the restored state may be older than the real infrastructure. Use terraform import to re-add any missing resources.

**Step 5 — Manual State Repair (If No Backup)**

If no backup exists (which should never happen in production), you must rebuild the state by reimporting every resource Terraform previously managed:

```
terraform import aws_vpc.main vpc-0a1b2c3d4e5f
terraform import aws_subnet.private_a subnet-0a1b2c3d
# ... repeat for every resource
```

This is extremely tedious for large environments. A complete state rebuild can take hours. This scenario reinforces why versioning on the backend is non-negotiable.

**Force-Unlock Stale Locks**

If the state is locked (from a crashed pipeline) and Terraform refuses to run, force-unlock:

```
terraform force-unlock <LOCK_ID>
```

> *Warning: Only force-unlock if you are certain no other process is running. Unlocking while another apply is in progress can corrupt the state further.*

**Prevention Checklist**

- Enable versioning on S3/Blob Storage backend — automatic rollback capability.
- Enable state locking — DynamoDB (AWS) or Blob leases (Azure). Always.
- Only apply through CI/CD pipelines — eliminates concurrent local applies.
- Back up state manually before high-risk operations: terraform state mv, terraform import, major module refactors.

> *Tip: Treat state corruption as an incident. Write a post-mortem, identify the root cause, and add a prevention control. The most common cause is missing state locking — verify locking is enabled in every environment.*

---

## Q2. How do you manage secrets in Terraform without hardcoding them?

**Answer**

Hardcoding secrets in Terraform code or variable files is a critical security risk — once in Git, a secret is permanently in the history. Multiple patterns exist to inject secrets safely.

**Pattern 1 — Environment Variables (Simplest)**

Terraform automatically reads environment variables prefixed with TF_VAR_. The secret is never in code or state:

```
# Set in CI/CD pipeline secrets or local shell
export TF_VAR_db_password='s3cur3P@ss'

# In Terraform — declare variable, no default
variable "db_password" {
  description = "RDS master password"
  type        = string
  sensitive   = true    # Redacts value from plan/apply output
}

resource "aws_db_instance" "main" {
  password = var.db_password
}
```

**Pattern 2 — AWS Secrets Manager / Azure Key Vault (Production Best Practice)**

Retrieve secrets dynamically at plan/apply time from a secret manager. The secret value never appears in code or variable files:

```
# AWS Secrets Manager
data "aws_secretsmanager_secret_version" "db_creds" {
  secret_id = "prod/rds/credentials"
}

locals {
  db_creds = jsondecode(data.aws_secretsmanager_secret_version.db_creds.secret_string)
}

resource "aws_db_instance" "main" {
  password = local.db_creds["password"]
}

# Azure Key Vault
data "azurerm_key_vault_secret" "db_password" {
  name         = "db-password"
  key_vault_id = data.azurerm_key_vault.main.id
}

resource "azurerm_postgresql_server" "main" {
  administrator_login_password = data.azurerm_key_vault_secret.db_password.value
}
```

**Pattern 3 — HashiCorp Vault**

For multi-cloud or on-premise scenarios, HashiCorp Vault provides dynamic secrets — credentials generated on demand that expire automatically:

```
provider "vault" {
  address = "https://vault.company.com"
}

data "vault_generic_secret" "db" {
  path = "secret/prod/database"
}

resource "aws_db_instance" "main" {
  password = data.vault_generic_secret.db.data["password"]
}
```

**Pattern 4 — Terraform Cloud / Enterprise Workspace Variables**

Store secrets as sensitive workspace variables in Terraform Cloud. They are encrypted at rest, never shown in UI after entry, and injected at runtime. Teams never see the value.

**What NOT to Do**

| Bad Practice | Why It's Dangerous |
|---|---|
| Hardcode in .tf file | Committed to Git permanently |
| Put in terraform.tfvars | Often committed accidentally |
| Default value in variable block | Exposed in plan output and state |
| Store in plaintext in state | State files accessible to all pipeline agents |

**Sensitive Variable Flag**

Always mark secret variables with sensitive = true. This prevents the value from appearing in plan/apply output, Terraform Cloud logs, and error messages:

```
variable "api_key" {
  type      = string
  sensitive = true
}
```

> *Warning: sensitive = true hides values in Terraform output but the value IS still stored in the state file in plaintext. Always encrypt your state backend and restrict access to state files.*

> *Tip: Never use git history to retrieve a rotated secret. If a secret was ever committed to Git — even once, even in a deleted file — treat it as compromised. Rotate it immediately.*

---

## Q3. How do you handle Terraform state file drift in a team environment? What are your backup strategies?

**Answer**

State drift occurs when the real infrastructure differs from what Terraform's state file records. In team environments, drift is inevitable — the goal is to detect it quickly and remediate safely.

**What Causes Drift in Teams**

- Manual changes — Engineer modifies a security group in the console to fix an urgent issue, forgets to reflect it in code.
- External automation — Another tool (Ansible, CloudFormation, AWS autoscaling) modifies a resource Terraform manages.
- Resource self-mutation — Some AWS resources change their own properties after creation (e.g., EKS updates endpoint configuration).
- Incomplete apply — A previous apply partially succeeded, leaving some resources in an intermediate state.

**Detecting Drift**

```
# Standard plan — shows all changes including drift
terraform plan

# Refresh-only — ONLY detects what changed in real infra, no code changes
terraform plan -refresh-only
terraform apply -refresh-only   # Updates state to match reality
```

The -refresh-only flag is the safest drift detection tool. It updates the state to match reality without making any infrastructure changes — it never modifies resources.

**Drift Detection in CI/CD**

Run a scheduled drift detection pipeline (nightly or every 6 hours) that runs terraform plan -refresh-only and alerts if drift is detected:

```
# GitHub Actions scheduled drift detection
on:
  schedule:
    - cron: '0 */6 * * *'   # Every 6 hours

jobs:
  detect-drift:
    steps:
    - run: terraform plan -refresh-only -detailed-exitcode
      id: plan
    - name: Alert on drift
      if: steps.plan.outputs.exitcode == '2'
      run: |
        curl -X POST $SLACK_WEBHOOK \
          -d '{"text":"Drift detected in prod Terraform state!"}'
```

terraform plan returns exit code 2 when changes are detected, 0 when no changes, 1 on error. This makes it scriptable.

**Remediating Drift**

| **Drift Type** | Remediation Approach |
|---|---|
| Manual change is valid — should be permanent | Update Terraform code to match reality, then plan (shows no diff) |
| Manual change is invalid — should be reverted | Run terraform apply — Terraform reverts to the desired state |
| Resource modified by external tool | Add lifecycle ignore_changes for that attribute, or take ownership in Terraform |
| Resource deleted manually | terraform import to re-add to state, or remove from code and reimport |

**Backup Strategies**

- Backend versioning (mandatory) — Enable S3 object versioning or Azure Blob versioning. Every state write creates a versioned copy. Restore in seconds.
- Pre-operation manual backup — Before high-risk operations (module refactors, mass rename, destroy), manually download the state:

```
# AWS
aws s3 cp s3://tfstate-bucket/prod/terraform.tfstate \
  ./terraform.tfstate.backup.$(date +%Y%m%d-%H%M%S)

# Azure
az storage blob download \
  --account-name tfstateaccount --container-name tfstate \
  --name prod/terraform.tfstate \
  --file ./terraform.tfstate.backup.$(date +%Y%m%d-%H%M%S)
```

- State snapshots in Terraform Cloud — Terraform Cloud and Enterprise automatically snapshot state on every apply with full history.
- Cross-region replication — Replicate the S3 state bucket to another region for disaster recovery.

**Team Governance to Prevent Drift**

- No manual console changes to Terraform-managed resources — enforce via IAM conditions or SCP policies.
- AWS Config rule — Alert when tagged Terraform-managed resources are modified outside of pipeline.
- Tag all resources with ManagedBy=Terraform and LastAppliedBy=<pipeline-run-id>.

> *Tip: Run terraform plan -refresh-only in your CD pipeline as a pre-step before every apply. If it shows unexpected changes, fail the pipeline and alert the team — someone may have made a manual change that would be overwritten.*

---

## Q4. What are the best practices for organizing Terraform code and managing multiple environments?

**Answer**

Terraform code organisation directly impacts maintainability, safety, and team productivity. The right structure prevents accidental changes between environments and makes code reusable.

**Recommended Directory Structure**

```
infrastructure/
├── modules/                    # Reusable, environment-agnostic modules
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── versions.tf
│   ├── eks-cluster/
│   ├── rds/
│   └── iam-role/
│
└── environments/               # Per-environment root modules
    ├── dev/
    │   ├── main.tf             # Calls modules with dev values
    │   ├── variables.tf
    │   ├── terraform.tfvars    # Dev-specific values
    │   ├── backend.tf          # Dev state backend config
    │   └── outputs.tf
    ├── staging/
    └── prod/
```

**Module Design Principles**

- Single responsibility — Each module does one thing (networking, EKS cluster, RDS instance). Do not create a monolithic 'everything' module.
- Interface via variables and outputs — Modules accept inputs via variables and expose results via outputs. No hardcoding of environment-specific values inside modules.
- Semantic versioning — Tag module releases. Environments pin to specific versions:

```
module "eks" {
  source  = "git::https://github.com/company/terraform-modules.git//eks?ref=v2.3.0"
  # or Terraform Registry:
  source  = "company/eks/aws"
  version = "~> 2.3"
  cluster_name   = var.cluster_name
  node_count     = var.node_count
}
```

**Environment Separation**

- Separate directories + separate state files — Each environment is an independent root module with its own backend. A terraform apply in dev cannot touch prod state.
- Use terraform.tfvars per environment — Different values for instance sizes, replica counts, retention periods:

```
# environments/dev/terraform.tfvars
cluster_name    = "eks-dev"
node_count      = 2
instance_type   = "t3.medium"
db_backup_days  = 7

# environments/prod/terraform.tfvars
cluster_name    = "eks-prod"
node_count      = 10
instance_type   = "m5.xlarge"
db_backup_days  = 35
```

**Naming Conventions**

- Resources: <service>-<environment>-<region> (e.g., eks-prod-useast1)
- Variables: snake_case (cluster_name, not clusterName)
- Modules: noun that describes the component (networking, not create-vpc)
- Files: main.tf (primary resources), variables.tf (inputs), outputs.tf (outputs), versions.tf (provider/Terraform version constraints)

**Provider and Version Pinning**

```
terraform {
  required_version = ">= 1.5, < 2.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"    # Allow patch updates, not major
    }
  }
}
```

**Code Review and CI Workflow**

- PR-based workflow — All Terraform changes via pull request. Pipeline runs terraform plan on PR and posts the plan output as a comment.
- terraform fmt and terraform validate in CI — Fail the pipeline if code is not properly formatted or has syntax errors.
- Automated plan review — Tools like Atlantis, Scalr, or Spacelift automate plan/apply through pull requests.

> *Tip: Avoid Terraform workspaces for environment separation in production. Workspaces share the same backend bucket and it is easy to accidentally apply to the wrong workspace. Separate directories with separate backends are safer and more explicit.*

---

## Q5. Design and implement a Terraform module for a multi-tier application infrastructure with proper state management.

**Answer**

A multi-tier application (web, application, database) implemented as composable Terraform modules demonstrates real-world module design, state isolation, and team-safe infrastructure management.

**Module Structure**

```
modules/
├── networking/     # VPC, subnets, NAT, IGW, security groups
├── compute/        # EC2 ASG / EKS cluster
├── database/       # RDS instance, subnet group, parameter group
└── loadbalancer/   # ALB, target groups, listeners
```

**Module 1 — Networking**

```
# modules/networking/main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = merge(var.common_tags, { Name = "${var.env}-vpc" })
}

resource "aws_subnet" "public" {
  for_each          = var.public_subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  tags = { Name = "${var.env}-public-${each.key}" }
}

# modules/networking/outputs.tf
output "vpc_id"            { value = aws_vpc.main.id }
output "public_subnet_ids" { value = values(aws_subnet.public)[*].id }
output "private_subnet_ids"{ value = values(aws_subnet.private)[*].id }
```

**Module 2 — Database**

```
# modules/database/main.tf
resource "aws_db_instance" "main" {
  identifier           = "${var.env}-postgres"
  engine               = "postgres"
  engine_version       = var.engine_version
  instance_class       = var.instance_class
  allocated_storage    = var.storage_gb
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  password             = var.db_password
  multi_az             = var.multi_az
  deletion_protection  = var.env == "prod" ? true : false
  skip_final_snapshot  = var.env == "prod" ? false : true
  tags = var.common_tags
}
```

**Root Module — Assembling Tiers**

```
# environments/prod/main.tf
module "networking" {
  source      = "../../modules/networking"
  env         = "prod"
  vpc_cidr    = "10.0.0.0/16"
```

```
  public_subnets  = { a = { cidr="10.0.1.0/24", az="us-east-1a" } }
  private_subnets = { a = { cidr="10.0.11.0/24", az="us-east-1a" } }
  common_tags = local.common_tags
}

module "database" {
  source          = "../../modules/database"
  env             = "prod"
  vpc_id          = module.networking.vpc_id
  subnet_ids      = module.networking.private_subnet_ids
  db_password     = data.aws_secretsmanager_secret_version.db.secret_string
  instance_class  = "db.m5.large"
  multi_az        = true
  common_tags     = local.common_tags
}
```

**State Management**

```
# environments/prod/backend.tf
terraform {
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "prod/us-east-1/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt        = true
  }
}
```

Each environment has its own backend key, its own DynamoDB lock entry, and its own state file. A deployment to dev can never affect prod state.

**Cross-State Data Sharing (remote_state)**

When one root module needs outputs from another (e.g., the compute module needs the VPC ID provisioned by the networking stack), use terraform_remote_state:

```
data "terraform_remote_state" "networking" {
  backend = "s3"
  config = {
    bucket = "company-terraform-state"
    key    = "prod/networking/terraform.tfstate"
    region = "us-east-1"
  }
}

# Use output from networking state
vpc_id = data.terraform_remote_state.networking.outputs.vpc_id
```

> *Tip: Split large environments into multiple state files per logical tier (networking, compute, database) rather than one giant state. This limits the blast radius of any single plan and speeds up operations significantly.*

---

## Q6. How do you prevent hardcoded secrets in Terraform and Dockerfiles?

**Answer**

Hardcoded secrets are the most common source of credential leaks in DevOps. Prevention requires controls at every layer — developer tooling, CI/CD scanning, and secret management integration.

**Terraform — Preventing Hardcoded Secrets**

- Never set a default value for sensitive variables — Force the caller to provide the value:

```
variable "db_password" {
  type      = string
  sensitive = true
  # No default — Terraform will error if not provided
}
```

- Use data sources for secrets — Retrieve from AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault at runtime (see Q2 for full examples).
- Mark sensitive outputs — Prevent secrets from appearing in output:

```
output "db_connection_string" {
  value     = "postgresql://${var.db_user}:${var.db_password}@$
{aws_db_instance.main.endpoint}"
```

```
  sensitive = true
}
```

**Dockerfile — Preventing Hardcoded Secrets**

- Never use ARG or ENV for secrets in Dockerfiles — These values are stored in image layers and visible in docker history:

```
# WRONG — Secret visible in 'docker history'
ARG DB_PASSWORD
ENV DB_PASSWORD=${DB_PASSWORD}

# CORRECT — Inject secrets at runtime, not build time
# Pass via docker run -e or Kubernetes secret
```

- Use multi-stage builds to avoid secrets in final image — Build with credentials in the builder stage, copy only the artifact:

```
FROM node:20 AS builder
# Mount secret during build — not stored in layer
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) npm install

FROM node:20-alpine AS final
COPY --from=builder /app/dist ./dist
# npm_token is NOT in the final image
```

- Use Docker BuildKit secrets — BuildKit mounts secrets as temporary files during the build step only:

```
DOCKER_BUILDKIT=1 docker build \
  --secret id=npm_token,src=~/.npmrc .
```

**Pre-Commit Detection — Stop Secrets Before They Enter Git**

- Install pre-commit hooks with detect-secrets or git-secrets:

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/Yelp/detect-secrets
  hooks:
  - id: detect-secrets
    args: ['--baseline', '.secrets.baseline']
```

**CI/CD Pipeline Scanning — Catch What Pre-Commit Misses**

```
# GitHub Actions — TruffleHog secret scanning
- name: Scan for secrets
  uses: trufflesecurity/trufflehog@main
  with:
    path: ./
    base: ${{ github.event.repository.default_branch }}
    extra_args: --only-verified
```

**Git History Remediation**

If a secret has already been committed, rotate it immediately (the secret is compromised regardless of whether you remove it). Then purge from history using BFG Repo Cleaner:

```
bfg --replace-text secrets.txt my-repo.git
git push --force
```

> *Warning: Removing a secret from Git history does not undo exposure. Anyone who pulled the repo before the purge has the secret. Rotate the credential immediately after discovery, before removing from history.*

> *Tip: Add a .gitignore entry for terraform.tfvars (it often contains sensitive defaults) and ensure your .tfvars files used in CI/CD are stored as encrypted CI/CD secrets, not committed to the repository.*

---

## Q7. How would you manage cross-region deployments using Terraform in a multi-cloud setup?

**Answer**

Cross-region and multi-cloud Terraform requires multiple provider configurations, separate state files per region/cloud, and a structured approach to code sharing.

**Multiple Provider Aliases for Multi-Region**

```
provider "aws" {
  alias  = "us_east"
  region = "us-east-1"
}

provider "aws" {
  alias  = "eu_west"
```

```
  region = "eu-west-1"
}

# Resource targeting a specific region
resource "aws_instance" "us_server" {
  provider      = aws.us_east
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
}

resource "aws_instance" "eu_server" {
  provider      = aws.eu_west
  ami           = "ami-0bcc094d503a71547"
  instance_type = "t3.medium"
}
```

**Multi-Cloud — AWS + Azure in One Configuration**

```
provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
  subscription_id = var.azure_subscription_id
}

resource "aws_s3_bucket" "backup" { bucket = "cross-cloud-backup" }

resource "azurerm_storage_account" "backup" {
  name                = "crosscloudbackup"
  resource_group_name = azurerm_resource_group.main.name
  location            = "East US"
  account_tier        = "Standard"
}
```

**State Organisation for Multi-Region**

```
state/
├── aws/
│   ├── us-east-1/terraform.tfstate
│   ├── eu-west-1/terraform.tfstate
│   └── ap-southeast-1/terraform.tfstate
└── azure/
    ├── eastus/terraform.tfstate
    └── westeurope/terraform.tfstate
```

Each region/cloud has its own state file and independent locking. A failure in one region does not block deployments to another.

**Cross-Region Data References**

When resources in one region depend on resources in another (e.g., an S3 bucket in us-east-1 that serves as a backup target for eu-west-1), use terraform_remote_state or SSM Parameter Store to share values:

```
# us-east-1 stack outputs the backup bucket ARN
output "backup_bucket_arn" { value = aws_s3_bucket.backup.arn }

# eu-west-1 stack reads it
data "terraform_remote_state" "us_east" {
  backend = "s3"
  config  = { bucket = "tf-state", key = "aws/us-east-1/terraform.tfstate", region = "us-
east-1" }
}

resource "aws_s3_bucket_replication_configuration" "main" {
  destination_bucket_arn = data.terraform_remote_state.us_east.outputs.backup_bucket_arn
}
```

**CI/CD Pipeline for Multi-Region**

Use a matrix strategy to deploy all regions in parallel, with a sequential fallback for dependent regions:

```
strategy:
  matrix:
    region: [us-east-1, eu-west-1, ap-southeast-1]
  fail-fast: false   # Don't abort other regions if one fails
```

*Tip: Use Terragrunt for multi-region deployments. Its run-all command and dependency management make it trivial to apply an entire multi-region, multi-cloud infrastructure in dependency order with a single command.*

---

## Q8. How would you refactor a legacy Terraform codebase used by multiple teams to follow best practices like DRY and modularity?

**Answer**

Refactoring a legacy Terraform codebase is high-risk work. The primary danger is that Terraform sees the refactored code as a resource deletion + recreation, causing production outages. A careful, incremental approach avoids this.

**Why Legacy Terraform Codebases Become Problematic**

- Copy-pasted resource blocks across environments — The same EC2 configuration repeated 10 times with minor differences.
- Flat structure — All resources in a single main.tf file with hundreds of resources.
- No module boundaries — Changes to one part of the code affect unrelated resources.
- Outdated provider versions — Pinned to old providers with security vulnerabilities.

**Step 1 — Audit Before Touching Anything**

```
terraform show -json | jq '.values.root_module.resources | length'
```

Map all resources: what exists, what module it's in, what depends on what. Use terraform graph | dot -Tsvg > graph.svg to visualise the dependency graph.

**Step 2 — Extract Modules Without Moving Resources**

The key insight: you can move resource definitions into a module WITHOUT recreating the resource, if you use moved blocks or terraform state mv to update the state reference.

```
# Before: resource defined at root level
resource "aws_security_group" "app" { ... }

# After: moved into a module
module "compute" {
  source = "./modules/compute"
}

# moved block — tells Terraform the resource moved, don't recreate
moved {
  from = aws_security_group.app
  to   = module.compute.aws_security_group.app
}
```

The moved block, introduced in Terraform 1.1, is the safe way to refactor. Without it, Terraform sees the old resource as deleted and the new module resource as new — it destroys and recreates in production.

**Step 3 — Eliminate Duplication with Modules**

Identify repeated patterns. Extract into a module with parameterised inputs:

```
# Before — 3 identical EC2 blocks with different names
resource "aws_instance" "web_a" { ami = "ami-xxx", instance_type = "t3.medium" }
resource "aws_instance" "web_b" { ami = "ami-xxx", instance_type = "t3.medium" }

# After — for_each over a module
module "web_servers" {
  source        = "./modules/ec2-server"
  for_each      = toset(["a", "b", "c"])
  instance_name = "web-${each.key}"
  instance_type = "t3.medium"
}
```

**Step 4 — Version and Release Modules**

Once modules are stable, move them to a dedicated repository and tag releases. Teams reference specific versions, decoupling module development from environment deployments.

**Step 5 — Upgrade Providers Incrementally**

Upgrade provider versions one minor version at a time. Run terraform plan after each upgrade and review the diff carefully — provider upgrades sometimes cause attribute changes that force resource recreation.

**Step 6 — Split Giant State Files**

If all resources are in one state, split into logical states (networking, compute, database). Use terraform state mv to move resources between states without recreation:

```
# Move resource to new state (run from new root module directory)
terraform state mv \
  -state=../old/terraform.tfstate \
  -state-out=./terraform.tfstate \
  aws_vpc.main aws_vpc.main
```

> *Warning: Always back up state before any refactoring operation. Test every refactor in a non-production environment first and validate with terraform plan that the diff shows no resource recreation before applying to production.*

> *Tip: Refactor in small, reviewable PRs rather than one giant change. A PR that moves 5 resources into a module and shows an empty terraform plan diff is safe to merge. A PR that changes 200 resources at once is a production risk.*

---

## Q9. Explain the internals of how Terraform handles dependencies and graph building during the planning phase.

**Answer**

Understanding Terraform's dependency graph is fundamental to writing correct, predictable Terraform code. It explains why resources are created in the right order and what happens when that order breaks.

**The Dependency Graph**

When you run terraform plan, Terraform parses all .tf files in the current directory and builds a Directed Acyclic Graph (DAG) of resources. Each node in the graph is a resource. Edges represent dependencies. Terraform creates resources in topological order — no resource is created before all its dependencies.

**Implicit Dependencies — How Terraform Infers Order**

Terraform automatically detects dependencies by tracking references between resources. When Resource B references an attribute of Resource A, Terraform knows A must be created before B:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "private" {
  vpc_id = aws_vpc.main.id   # ← implicit dependency on aws_vpc.main
  cidr_block = "10.0.1.0/24"
}
```

Terraform sees vpc_id = aws_vpc.main.id and records: 'aws_subnet.private depends on aws_vpc.main'. The VPC is always created first, then the subnet.

**How the Graph Is Built**

- Step 1: Parse — All .tf files parsed into an internal AST (Abstract Syntax Tree).
- Step 2: Evaluate expressions — Each attribute value is evaluated. References like aws_vpc.main.id create graph edges.
- Step 3: Build DAG — All resources become nodes. All references become directed edges (A → B means B depends on A).
- Step 4: Topological sort — Terraform sorts the graph into a valid creation order. Resources with no dependencies are at the root.
- Step 5: Parallelism — Terraform walks the graph and applies independent resources in parallel (default: 10 concurrent operations). Resources are only started when all their dependencies are complete.

**Explicit Dependencies — depends_on**

Sometimes a dependency cannot be expressed through attribute references. depends_on forces an explicit dependency:

```
resource "aws_iam_role_policy" "example" { ... }

resource "aws_lambda_function" "my_lambda" {
  depends_on = [aws_iam_role_policy.example]
  # Lambda needs the policy attached before it can execute
  # but the policy ARN is not used in any attribute
}
```

depends_on should be a last resort because it creates an opaque dependency — the graph has an edge but no attribute reference to explain why. It also disables Terraform's ability to optimise the order.

**Destroy Order — Reverse Graph**

On terraform destroy, Terraform reverses the graph. Resources with no dependants are destroyed first. This ensures you never destroy a VPC before the subnets inside it are destroyed.

**Viewing the Graph**

```
terraform graph | dot -Tsvg > graph.svg
```

This generates a visual SVG of the dependency graph using Graphviz. Extremely useful for debugging ordering issues in complex configurations.

**When the Graph Goes Wrong — Cycles**

A circular dependency (A depends on B, B depends on A) is a cycle. Terraform detects cycles and refuses to plan:

```
Error: Cycle: aws_security_group.a, aws_security_group.b
```

Fix cycles by breaking the circular reference — usually by using a separate aws_security_group_rule resource instead of inline ingress/egress rules.

> *Tip: Terraform parallelises independent resources by default. If your apply seems slow, run terraform apply -parallelism=20 to increase concurrent operations. Decrease it to 1 for debugging ordering issues.*

---

## Q10. How do you manage and isolate Terraform state files across multiple environments and teams?

**Answer**

State isolation is a safety requirement in team environments. Without it, a mistake in one environment can affect another, and concurrent operations cause corruption.

**Isolation by Directory + Backend Key**

The safest pattern: each environment is a separate root module directory with its own backend configuration pointing to a unique state key:

```
# environments/dev/backend.tf
terraform { backend "s3" { key = "dev/terraform.tfstate" } }

# environments/prod/backend.tf
terraform { backend "s3" { key = "prod/terraform.tfstate" } }
```

A developer running terraform apply in environments/dev cannot affect environments/prod — they are physically separate state files with separate lock entries.

**Isolation by AWS Account / Azure Subscription**

The strongest isolation: each environment lives in a completely separate AWS account or Azure subscription. The state bucket is in a central management account and uses separate prefixes per account:

```
company-tfstate-bucket/
├── management/terraform.tfstate
├── dev-account/terraform.tfstate
├── staging-account/terraform.tfstate
└── prod-account/terraform.tfstate
```

**Team Isolation — Per-Team State**

In large organisations, different teams manage different parts of infrastructure. Separate state per team prevents cross-team interference:

```
tfstate/
├── platform-team/
│   ├── networking/terraform.tfstate
│   └── eks-cluster/terraform.tfstate
├── data-team/
│   └── data-lake/terraform.tfstate
└── app-team/
    └── application/terraform.tfstate
```

**Access Control on State Files**

- AWS S3: Use IAM policies to restrict S3 key prefixes per team. The platform team's IAM role can only access platform-team/* keys.
- Azure Blob: Use separate storage containers per team with RBAC assignments per container.
- Terraform Cloud: Use separate workspaces per team with team-level access controls.

```
# IAM policy — restrict to team prefix
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::tfstate-bucket/platform-team/*"
}
```

**Workspaces — Limited Use Case**

Terraform workspaces create separate state files within the same backend. They work for managing minor variations but share the same code, which makes large environment differences awkward:

```
terraform workspace new staging
terraform workspace select prod
# Each workspace has its own state: terraform.tfstate.d/prod/terraform.tfstate
```

Workspaces are suitable for: feature branch environments, short-lived test environments, or small teams with very similar environments. They are NOT recommended for dev/staging/prod isolation in large teams.

**State Data Shared Across Teams — remote_state**

When the app team needs the VPC ID that the platform team manages, use terraform_remote_state with read-only access:

```
data "terraform_remote_state" "networking" {
  backend = "s3"
  config  = { bucket = "tfstate", key = "platform-team/networking/terraform.tfstate" }
}
# App team reads platform outputs, cannot modify platform state
```

> *Tip: Give teams read access to each other's state outputs but write access only to their own. This enables cross-team data sharing (VPC IDs, cluster endpoints) without giving teams the ability to modify each other's infrastructure.*

---

## Q11. What's your strategy to prevent and recover from a corrupted or deleted remote backend state file?

**Answer**

State file loss is a serious incident. Recovery depends entirely on your prevention controls. The strategy is: make loss impossible first, and recovery fast if it happens anyway.

**Prevention — Make Deletion Impossible**

- Enable S3 Object Versioning — Every terraform apply creates a new version. You can retrieve any previous state. This is mandatory for production.
- Enable S3 Object Lock (WORM mode) — Prevents deletion even by administrators:

```
aws s3api put-object-lock-configuration \
  --bucket my-tfstate-bucket \
  --object-lock-configuration \
  '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":
{"Mode":"GOVERNANCE","Days":90}}}'
```

- Enable MFA Delete on S3 — Requires physical MFA device to delete objects or disable versioning. Prevents accidental or malicious deletion.
- Restrict DeleteObject permissions — Remove s3:DeleteObject from the Terraform pipeline's IAM role. The pipeline only needs GetObject, PutObject, and ListBucket:

```
{
  "Effect": "Deny",
  "Action": ["s3:DeleteObject", "s3:DeleteBucket"],
  "Resource": "arn:aws:s3:::my-tfstate-bucket/*"
}
```

- Replicate to another region — Enable S3 Cross-Region Replication for a geographically isolated copy:

```
aws s3api put-bucket-replication \
  --bucket my-tfstate-bucket \
  --replication-configuration file://replication-config.json
```

**Detection — Know Before Users Do**

- CloudTrail alert on DeleteObject for the state bucket — immediate SNS notification to the team.
- S3 Event Notification on delete events → Lambda → PagerDuty.

```
# CloudWatch metric filter on CloudTrail
filter pattern: '{$.eventName = DeleteObject && $.requestParameters.bucketName = "my-
tfstate-bucket"}'
```

**Recovery — S3 Versioning Restore**

```
# List all versions of the state file
aws s3api list-object-versions \
  --bucket my-tfstate-bucket \
  --prefix prod/terraform.tfstate

# Restore previous version (copy it as new current version)
aws s3api copy-object \
  --copy-source my-tfstate-bucket/prod/terraform.tfstate?versionId=<VERSION_ID> \
  --bucket my-tfstate-bucket \
```

```
  --key prod/terraform.tfstate

# Validate
terraform init && terraform plan
```

**Recovery — No Backup Available (Worst Case)**

If no backup exists, you must rebuild the state by importing every resource. This is extremely time-consuming for large environments:

```
# Create empty state
terraform init

# Import each resource individually
terraform import aws_vpc.main vpc-0a1b2c3d
terraform import aws_subnet.private_a subnet-0a1b2c3d
# ... hundreds of resources ...

# Alternative: use Terraformer to auto-generate imports
terraformer import aws --resources=vpc,subnet,eks,rds --regions=us-east-1
```

**Terraform Cloud — Built-In State History**

Terraform Cloud maintains a full audit trail of every state version with one-click restore. If you use Terraform Cloud, state loss is nearly impossible without deliberate action.

> *Tip: Test your recovery process quarterly. Intentionally corrupt a dev state file and practice restoring it. Recovery under pressure during a real incident is not the time to discover your restore process doesn't work.*

---

## Q12. Have you implemented policy-as-code (e.g., Sentinel, OPA) with Terraform? Give a real use case.

**Answer**

Policy-as-Code enforces organisational rules on Terraform plans before resources are created. It catches misconfigurations, security violations, and cost issues automatically — without requiring peer reviewers to know every rule.

**Two Main Tools**

| Tool | Platform |
|---|---|
| HashiCorp Sentinel | Terraform Cloud/Enterprise |
| OPA (Open Policy Agent) | Any CI/CD system |

**Real Use Case 1 — Enforce Approved AWS Regions**

Policy: All AWS resources must be in us-east-1 or eu-west-1. No resources can be created in unapproved regions.

```
# OPA Policy — rego
package terraform.rules.approved_regions

approved_regions := {"us-east-1", "eu-west-1"}

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  resource.type == "aws_instance"
  not approved_regions[resource.values.availability_zone]
  msg := sprintf("EC2 instance '%v' is in unapproved region", [resource.address])
}

# Run in CI pipeline
terraform show -json tfplan.binary > tfplan.json
conftest test tfplan.json --policy ./policies/
```

**Real Use Case 2 — Require Tags on All Resources**

```
# Sentinel policy — require mandatory tags
import "tfplan/v2" as tfplan

required_tags = ["Environment", "Team", "ManagedBy"]

violations = [resource
  for _, resources in tfplan.resource_changes
  for resource in resources
  if resource.change.actions contains "create"
  if not all_tags_present(resource.change.after.tags)
```

```
]

all_tags_present(tags) {
  all required_tags as tag { tags[tag] }
}

main = rule { length(violations) == 0 }
```

**Real Use Case 3 — Prevent Publicly Accessible S3 Buckets**

```
# OPA — block any S3 bucket with public access enabled
deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  resource.type == "aws_s3_bucket_public_access_block"
  resource.values.block_public_acls == false
  msg := sprintf("S3 bucket '%v' has public access enabled — denied", [resource.address])
}
```

**Real Use Case 4 — Instance Type Governance**

```
# Sentinel — only approved instance types in production
approved_instance_types = ["t3.medium", "m5.large", "m5.xlarge"]

deny_unapproved_instances = rule {
  all tfplan.resource_changes["aws_instance"] as _, instances {
    all instances as instance {
      instance.change.after.instance_type in approved_instance_types
    }
  }
}
```

**Integrating OPA into Azure DevOps / GitHub Actions**

```
# GitHub Actions pipeline step
- name: OPA Policy Check
  run: |
    terraform show -json tfplan > tfplan.json
    conftest test tfplan.json \
      --policy ./policies/ \
      --output tap
  # Pipeline fails if any deny rule matches
```

*Tip: Start with 3-5 high-value policies (required tags, approved regions, no public S3) rather than trying to encode every rule at once. Add policies incrementally as violations are discovered in code reviews.*

---

## Q13. Explain the differences between terraform plan, terraform refresh, and terraform apply. When should each be used?

**Answer**

These three commands serve distinct purposes in the Terraform workflow. Understanding when to use each prevents accidental changes and makes debugging faster.

**terraform plan**

Reads the current state file, calls the provider APIs to refresh resource attributes, compares desired configuration (code) vs actual state, and shows what changes WOULD be made — without making any changes.

- Use when: reviewing a PR, validating a change before applying, understanding the impact of a configuration change.
- Safe to run anytime — it is read-only for infrastructure. It does write to the state file if -refresh=true (default).

```
terraform plan                  # Standard plan
terraform plan -out=tfplan.bin   # Save plan for later apply (CI/CD)
terraform plan -target=aws_ec2_instance.web   # Plan only one resource
terraform plan -refresh=false    # Skip API refresh (faster, uses cached state)
```

**terraform refresh**

Queries all provider APIs for the actual current state of every managed resource and updates the local state file to match reality — WITHOUT comparing to code or planning any changes.

- Use when: you suspect the state is out of date and want to sync it with reality before planning.
- Deprecated in favour of terraform plan -refresh-only — which shows what would change before making changes.

```
# Modern equivalent (shows changes before applying them)
terraform plan -refresh-only
terraform apply -refresh-only   # Updates state to match reality
```

**terraform apply**

Executes the plan — creates, updates, or destroys resources to match the desired configuration. Prompts for confirmation unless -auto-approve is passed.

- Use when: you have reviewed the plan output and want to make the changes.
- Always review plan output before apply. In CI/CD, save the plan with -out=tfplan.bin and apply the saved plan to ensure what was reviewed is exactly what gets applied.

```
terraform apply                   # Plan + apply (prompts for confirmation)
terraform apply tfplan.bin        # Apply a saved plan (no re-planning)
terraform apply -auto-approve     # Skip confirmation (CI/CD pipelines)
terraform apply -target=aws_s3_bucket.logs   # Apply only one resource
```

**Comparison Table**

| Command | Reads State | Calls APIs |
|---|---|---|
| plan | Yes | Yes (refresh) |
| plan -refresh-only | Yes | Yes |
| apply -refresh-only | Yes | Yes |
| apply | Yes | Yes |
| refresh (deprecated) | Yes | Yes |

**CI/CD Best Practice — Plan then Apply**

In CI/CD, always save the plan and apply the saved file. This ensures what was reviewed in the plan step is exactly what gets applied — no re-planning between stages:

```
# Stage 1 — Plan
terraform plan -out=tfplan.bin
# Human reviews the plan output

# Stage 2 — Apply (uses exact same plan)
terraform apply tfplan.bin    # No re-planning; no prompt
```

> *Tip: Never use terraform apply without reviewing terraform plan first in production. In automated pipelines, always use the two-step plan-then-apply pattern to prevent unreviewed changes from reaching production.*

---

## Q14. How do you modularize Terraform for large-scale projects? Give examples.

**Answer**

Modularisation transforms Terraform from a collection of resource definitions into a composable infrastructure library. Well-designed modules are the difference between maintainable at 10 resources and maintainable at 10,000 resources.

**What Makes a Good Module**

- Single responsibility — One module does one thing: vpc creates the VPC and its direct dependencies. It does not also create EKS.
- Stable interface — Variables are the module's public API. Changes to variable names are breaking changes. Design interfaces carefully upfront.
- Sensible defaults — Modules should work with minimal input. Override defaults to customise. This makes modules easy to use in dev:

```
variable "instance_type" {
  default = "t3.medium"   # Works for dev out of the box
}
```

**Module File Structure**

```
modules/eks-cluster/
├── main.tf        # Primary resource definitions
├── variables.tf   # All input variables with descriptions
├── outputs.tf     # Values consumers need
├── versions.tf    # Provider version constraints
├── data.tf        # Data sources only
└── README.md      # Usage examples, inputs, outputs table
```

**Example — EKS Cluster Module**

```
# modules/eks-cluster/variables.tf
variable "cluster_name"  { type = string }
variable "k8s_version"   { type = string; default = "1.30" }
variable "node_groups"   {
```

```
  type = map(object({
    instance_types = list(string)
    min_size       = number
    max_size       = number
    desired_size   = number
  }))
}
variable "subnet_ids"    { type = list(string) }
variable "tags"          { type = map(string); default = {} }

# modules/eks-cluster/outputs.tf
output "cluster_endpoint"        { value = aws_eks_cluster.main.endpoint }
output "cluster_name"            { value = aws_eks_cluster.main.name }
output "node_group_role_arn"     { value = aws_iam_role.node_group.arn }
```

**Consuming the Module**

```
# environments/prod/main.tf
module "eks" {
  source  = "git::https://github.com/company/tf-modules.git//eks-cluster?ref=v3.2.0"
  cluster_name = "prod-eks"
  subnet_ids   = module.networking.private_subnet_ids
  node_groups  = {
    general = { instance_types=["m5.xlarge"], min_size=3, max_size=20, desired_size=5 }
    spot    = { instance_types=["m5.large","m5.xlarge"], min_size=0, max_size=50,
desired_size=0 }
  }
  tags = local.common_tags
}
```

**Module Composition — Modules Calling Modules**

A higher-level module can compose lower-level modules. A 'full-stack' module might internally call the networking module, the EKS module, and the RDS module:

```
# modules/full-stack/main.tf
module "networking" {
  source   = "../networking"
  vpc_cidr = var.vpc_cidr
}

module "eks" {
  source     = "../eks-cluster"
  subnet_ids = module.networking.private_subnet_ids
}
```

**Module Registry and Versioning**

- Terraform Registry — Public or private registry for module distribution. Modules are versioned and teams reference specific versions.
- Git tags — Tag module releases: v1.0.0, v1.1.0. Pin environments to specific tags. Only upgrade deliberately.
- Changelog — Maintain a CHANGELOG.md in the module repo. Mark breaking changes (major version bump) vs new features (minor) vs bug fixes (patch).

> *Tip: Create a module template repository with the standard file structure, README format, and example usage. Every new module starts from this template — ensures consistency across all modules in the organisation.*

---

## Q15. How do you detect and resolve drift in Terraform-managed resources?

**Answer**

Drift is the gap between what Terraform's state says should exist and what actually exists in the cloud. Left undetected, drift causes apply failures, security gaps, and unexpected resource changes.

**Types of Drift**

| Drift Type | Description |
|---|---|
| State drift | Real infrastructure differs from state |
| Configuration drift | State differs from Terraform code |

| | |
|---|---|
| Silent mutation | Resource changed itself |

**Detection Method 1 — terraform plan -refresh-only**

```
terraform plan -refresh-only
```

Terraform calls every provider API to get the current real-world state of each resource and shows what changed. It does NOT compare to your code — only state vs reality.

```
# Output shows drift:
~ aws_security_group.app
  ~ ingress {
    + cidr_blocks = ['10.0.0.0/8']  # Added manually — not in state
  }
```

**Detection Method 2 — terraform plan (Full Plan)**

```
terraform plan
```

A full plan does both: refreshes state from APIs AND compares to code. Shows all differences between desired (code) and actual (cloud). This catches both state drift and configuration drift simultaneously.

**Detection Method 3 — Scheduled Drift Detection Pipeline**

Automate drift detection on a schedule. Alert the team when drift is found:

```
# GitLab CI — nightly drift detection
drift-detection:
  schedule: '0 2 * * *'
  script:
    - terraform init
    - terraform plan -refresh-only -detailed-exitcode
  after_script:
    - |
      if [ $? -eq 2 ]; then
        curl -X POST $PAGERDUTY_WEBHOOK \
          -d '{"message":"Terraform drift detected in production"}'
      fi
```

**Resolution — Choose Your Approach**

**Option A — Terraform Wins (Restore Desired State)**

If the manual change is invalid and should be reverted, run terraform apply. Terraform overwrites the reality with the desired configuration:

```
terraform plan    # Review what will change
terraform apply   # Revert to desired state
```

**Option B — Reality Wins (Update Code)**

If the manual change is valid and should become the new desired state, update the code to match what exists, then plan — the diff should disappear:

```
# Update code to match reality
# Then confirm plan shows no changes
terraform plan    # Should show: No changes
```

**Option C — Ignore the Drift**

If an attribute is managed by another process (autoscaler changes desired_capacity), tell Terraform to ignore changes to that attribute:

```
resource "aws_autoscaling_group" "main" {
  desired_capacity = 5
  lifecycle {
    ignore_changes = [desired_capacity]   # Managed by autoscaler
  }
}
```

**Prevention**

- Enforce no-console-change policies using SCPs, IAM permission boundaries, or Azure Policy.
- Tag all Terraform-managed resources with ManagedBy=Terraform and alert on changes to tagged resources made outside of Terraform.
- Use AWS Config rules to detect and auto-remediate configuration drift for specific resource types.

> *Tip: Run terraform plan -refresh-only in your CI/CD pipeline before every apply. If it shows unexpected changes, pause and investigate before proceeding — someone may have made a manual change that your apply will overwrite.*

---

## Q16. Show a use-case for for_each vs count in a real Terraform deployment. When should dynamic blocks be preferred?

**Answer**

for_each and count both create multiple instances of a resource, but they differ fundamentally in how Terraform identifies each instance in state — with significant implications for safety and maintainability.

**count — Index-Based Identification**

count creates N copies of a resource, identified by numeric index: resource.aws_subnet.private[0], [1], [2]. The problem: if you remove an element from the middle of the list, Terraform reindexes all subsequent elements and plans to recreate them.

```
# RISKY with count — removing "b" forces recreating "c"
variable "subnets" { default = ["a", "b", "c"] }

resource "aws_subnet" "private" {
  count      = length(var.subnets)
  cidr_block = cidrsubnet("10.0.0.0/16", 8, count.index)
  tags       = { Name = "subnet-${var.subnets[count.index]}" }
}
# State: aws_subnet.private[0], [1], [2]
# Remove "b": Terraform sees [0]=a, [1]=c — destroys [2] and updates [1]!
```

**for_each — Key-Based Identification**

for_each creates instances identified by a string key from a map or set. Removing one element only affects that element — other instances are untouched.

```
variable "subnets" {
  default = {
    "app-a" = { cidr = "10.0.11.0/24", az = "us-east-1a" }
    "app-b" = { cidr = "10.0.12.0/24", az = "us-east-1b" }
    "app-c" = { cidr = "10.0.13.0/24", az = "us-east-1c" }
  }
}

resource "aws_subnet" "private" {
  for_each          = var.subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  tags              = { Name = "${var.env}-${each.key}" }
}
# State: aws_subnet.private["app-a"], ["app-b"], ["app-c"]
# Remove "app-b": only aws_subnet.private["app-b"] is destroyed. Others untouched.
```

**When to Use count**

- When you need a fixed number of identical resources with no meaningful key (e.g., create 3 identical worker nodes).
- When the resources are truly interchangeable — removing any one of them and reindexing the others is acceptable.

```
resource "aws_instance" "workers" {
  count         = 3
  instance_type = "t3.medium"
  ami           = data.aws_ami.amazon_linux.id
}
```

**When to Use for_each**

- When resources have distinct identities (IAM users, S3 buckets, subnets, security group rules).
- When you may add or remove individual elements without affecting others.
- Almost always preferred over count for production resources.

**dynamic Blocks — For Repeated Nested Blocks Within One Resource**

dynamic blocks are NOT for creating multiple resources. They create multiple nested configuration blocks within a single resource — for attributes like ingress rules, policy statements, or environment variables:

```
resource "aws_security_group" "app" {
  name   = "app-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
```

```
      cidr_blocks = ingress.value.cidrs
    }
  }
}

# variable — drives the dynamic block
variable "ingress_rules" {
  default = [
    { port=80,  protocol="tcp", cidrs=["0.0.0.0/0"] },
    { port=443, protocol="tcp", cidrs=["0.0.0.0/0"] },
    { port=8080,protocol="tcp", cidrs=["10.0.0.0/8"] }
  ]
}
```

| Use Case | Right Tool |
|---|---|
| Multiple similar resources (subnets, IAM roles) | for_each with a map |
| Fixed number of identical resources | count |
| Multiple blocks inside one resource (ingress rules, env vars) | dynamic block |
| Check if something should exist at all | count = condition ? 1 : 0 |

> *Tip: Convert count to for_each in existing code using moved blocks. Add moved { from=resource[0]; to=resource["key"] } for each element. Terraform will update the state keys without recreating resources.*

---

## Q17. Explain the purpose and real-world use of lifecycle blocks (prevent_destroy, create_before_destroy) in Terraform.

**Answer**

The lifecycle block controls how Terraform creates, updates, and destroys individual resources. These meta-arguments override Terraform's default behaviour and are essential for zero-downtime deployments and protecting critical resources.

**prevent_destroy**

Prevents Terraform from destroying the resource even if the plan calls for it. Terraform will error instead of destroying:

```
resource "aws_db_instance" "production" {
  identifier = "prod-postgres"
  # ...
  lifecycle {
    prevent_destroy = true
  }
}

# When someone runs: terraform destroy
# Error: Instance cannot be destroyed
# on main.tf line 1: Resource aws_db_instance.production has
# lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed.
```

Real-world use cases:

- Production RDS instances, ElastiCache clusters, S3 buckets with data — accidental deletion is catastrophic.
- KMS keys — Once deleted, encrypted data is permanently inaccessible.
- DNS zones — Deletion causes immediate service outage.

> *Warning: prevent_destroy only prevents destruction within Terraform. Removing the resource from your .tf file OR removing the lifecycle block, then running destroy, will still delete it. It's a safeguard, not a hard lock.*

**create_before_destroy**

By default, Terraform destroys the old resource before creating the new one when a replacement is needed. This causes downtime. create_before_destroy reverses this: create the new resource first, then destroy the old one.

```
resource "aws_launch_template" "app" {
  name_prefix   = "app-lt-"
  image_id      = var.ami_id
  instance_type = var.instance_type

  lifecycle {
    create_before_destroy = true
  }
```

```
}

resource "aws_autoscaling_group" "app" {
  launch_template { id = aws_launch_template.app.id }
  # ASG uses the launch template; destroy order matters
}
```

Real-world use cases:

- Launch templates and Auto Scaling Groups — New launch template created first, ASG updated, old template deleted.
- TLS certificates (ACM) — New certificate created and validated before swapping ALB listener to use it.
- Security groups — Create new SG, update resources to use it, delete old SG.
- Any resource where a gap in availability causes a service disruption.

**ignore_changes**

Tells Terraform to ignore specific attribute changes in future plans. Once the attribute is set at creation, Terraform never proposes changes to it — even if the code or real value differs:

```
resource "aws_autoscaling_group" "app" {
  desired_capacity = 5
  lifecycle {
    ignore_changes = [desired_capacity, tags]
  }
}
```

Real-world use cases:

- desired_capacity on ASG — The autoscaler changes this dynamically. Terraform should not reset it on every apply.
- tags — Another tool manages tags. Terraform should not overwrite them.
- user_data on EC2 — Content changes shouldn't force instance replacement.

> *Warning: ignore_changes can silently hide real problems. If a critical security attribute is in ignore_changes, a misconfiguration introduced manually will never be detected or corrected. Use sparingly.*

**replace_triggered_by (Terraform 1.2+)**

Forces replacement of a resource when another referenced resource changes:

```
resource "aws_instance" "app" {
  lifecycle {
    replace_triggered_by = [aws_launch_template.app.id]
  }
}
```

> *Tip: Use prevent_destroy on every stateful production resource: RDS, ElastiCache, S3, DynamoDB, KMS keys. The cost of an accidental deletion (hours of recovery, data loss) far outweighs the minor inconvenience of removing the lifecycle block when a deletion is intentional.*

---

## Q18. What's the benefit of using for_each over count in complex Terraform modules?

**Answer**

This question builds on Q16. The core benefit of for_each over count in complex modules is stable resource identity — changes to the collection only affect the changed resources, not the entire collection.

**The Fundamental Problem with count in Modules**

When a module uses count, Terraform identifies each module instance as module.name[0], module.name[1], etc. If the list changes, every instance after the removal point gets a new index and Terraform plans to destroy and recreate it:

```
# Module call with count — DANGEROUS
module "microservice" {
  count  = length(var.services)
  name   = var.services[count.index]
  source = "./modules/service"
}

# var.services = ["auth", "payments", "orders"]
# State: module.microservice[0] module.microservice[1] module.microservice[2]
# Remove "payments" from middle:
# [0]=auth (unchanged), [1]=orders (UPDATE — was payments!), [2] DESTROY
```

Terraform sees index [1] as a different resource — plans to destroy the payments service and recreate an orders service in its place. This is dangerous for production.

**for_each in Modules — Stable Identity**

```
# Module call with for_each — SAFE
```

```
module "microservice" {
  for_each = toset(var.services)
  name     = each.key
  source   = "./modules/service"
}

# State: module.microservice["auth"] module.microservice["payments"]
module.microservice["orders"]
# Remove "payments":
# module.microservice["payments"] DESTROY (correct!)
# module.microservice["auth"] — untouched
# module.microservice["orders"] — untouched
```

**Benefits in Complex Modules**

- Partial destruction — Remove one service, EKS node group, or IAM role without affecting the others.
- Clearer state — module.service['payments'] is immediately understandable. module.service[2] requires counting the list.
- Safe refactoring — Reorder the collection in code without triggering recreations.
- Map-based configuration — each module instance can have its own configuration via a map:

```
variable "environments" {
  default = {
    dev     = { size = "t3.small",  replicas = 1 }
    staging = { size = "t3.medium", replicas = 2 }
    prod    = { size = "m5.large",  replicas = 5 }
  }
}

module "environment" {
  for_each      = var.environments
  env_name      = each.key
  instance_type = each.value.size
  replica_count = each.value.replicas
  source        = "./modules/environment"
}
```

**Migrating from count to for_each Safely**

When migrating an existing module from count to for_each, use moved blocks to update state keys without destroying resources:

```
moved {
  from = module.microservice[0]
  to   = module.microservice["auth"]
}
moved {
  from = module.microservice[1]
  to   = module.microservice["payments"]
}
```

> *Tip: Design all new modules with for_each in mind. If a module will ever be called multiple times, use for_each from day one. Migrating from count to for_each in production requires careful use of moved blocks and thorough plan review.*

---

## Q19. Why does terraform plan show changes when no code was modified? What's the difference between real drift, computed values, and provider behavior?

**Answer**

Seeing unexpected changes in terraform plan when you haven't changed any code is one of the most confusing Terraform experiences. Three distinct mechanisms cause this — each has a different root cause and fix.

**Cause 1 — Real Drift (Infrastructure Changed Outside Terraform)**

Someone modified the real infrastructure (console, CLI, another tool) after the last apply. Terraform refreshes from the API and sees a mismatch between state and reality:

```
# Someone added an inbound rule to the security group manually
~ aws_security_group.app
  + ingress { from_port=22, cidr=['0.0.0.0/0'] }   # Added manually
```

Fix: Either update the code to include the change (if valid) or apply to revert it (if invalid). This is true drift — the infrastructure changed.

**Cause 2 — Computed Values (Provider Calculates Attribute)**

Some attributes are computed by AWS/Azure after resource creation. The provider calculates a new value on every plan, causing a perpetual diff even though nothing really changed:

```
# Example: AWS adds the AWS account ID to a policy at apply time
~ aws_s3_bucket_policy.main
  ~ policy = jsonencode({...AccountID: "123456789"})
  → policy = jsonencode({...AccountID: "123456789"})
```

This happens because the local value (before apply) differs slightly from what AWS normalises and stores. Fix: Use data.aws_iam_policy_document instead of jsonencode — it produces the exact format AWS expects.

**Cause 3 — Provider Behaviour Changes**

When you upgrade a provider version, it may change how it reads or calculates attributes — causing new diffs on resources that haven't changed:

```
# After upgrading hashicorp/aws from 4.x to 5.x
~ aws_ecs_task_definition.main
  ~ container_definitions = "..." → "..."  # Provider now normalises JSON differently
```

Fix: Review provider changelog before upgrading. Use lifecycle { ignore_changes = [container_definitions] } for attributes that the provider normalises but that don't affect real behaviour. Pin provider versions to avoid surprise upgrades.

**Cause 4 — Non-Deterministic Configuration**

Using functions that produce different values on each plan (timestamp(), uuid()) causes perpetual diffs:

```
resource "aws_ssm_parameter" "build_time" {
  value = timestamp()   # Different every plan!
}
```

Fix: Avoid non-deterministic functions in resource arguments. Use them only in locals or outputs where they won't cause resource churn.

**How to Diagnose**

```
# See exactly what the provider is detecting
terraform plan -refresh-only

# Check provider logs for debugging
TF_LOG=DEBUG terraform plan 2>&1 | grep -A5 'RequestPlanProviderMeta'
```

| Symptom | Root Cause |
|---|---|
| Security group rules appeared | Manual console change (drift) |
| Policy diff despite same value | Provider JSON normalisation |
| Diff after provider upgrade | Provider attribute calculation changed |
| timestamp() diff every plan | Non-deterministic function |

> *Tip: Run terraform plan -refresh-only first when you see unexpected changes. If it shows diffs, the infrastructure changed. If the full plan shows diffs but refresh-only doesn't, the diff is from code/provider — not real infrastructure change.*

---

## Q20. How does Terraform decide between in-place update vs full resource replacement? Why can one small attribute destroy production?

**Answer**

Terraform's decision between update-in-place and destroy-then-recreate (replacement) is determined by the provider. Each attribute is annotated by the provider as either 'updateable' or 'forces new resource'. This is one of the most dangerous aspects of Terraform — a single attribute change can trigger recreation of a production database.

**How Providers Annotate Attributes**

Provider developers mark each attribute with a change behaviour. When you change a 'forces new resource' attribute, the provider tells Terraform: 'this resource cannot be updated in place; you must destroy and create a new one.'

```
# In terraform plan output:
~ resource "aws_db_instance" "main" will be updated in-place
  ~ allocated_storage = 100 → 200   # In-place: can be updated live

-/+ resource "aws_db_instance" "main" must be replaced  ← DANGER
  ~ identifier = "prod-db" → "prod-db-v2"   # Forces replacement
  # Terraform will DESTROY the production database and create a new one
```

**Common Attributes That Force Replacement**

| Resource | Attribute |
|---|---|
| aws_db_instance | identifier |
| aws_db_instance | engine |
| aws_instance | ami |
| aws_instance | subnet_id |
| aws_eks_cluster | name |
| aws_elasticache_cluster | engine |
| aws_iam_user | name |

**How to Check Before Applying**

Always carefully read the plan output. The critical symbols to look for:

```
~    Resource will be updated in-place (safe for most attributes)
-/+  Resource will be destroyed then recreated (DANGEROUS — check carefully)
+    New resource will be created
-    Resource will be destroyed
<=   Data source will be read
```

**Preventing Accidental Replacement**

- Use prevent_destroy lifecycle on critical resources — Terraform errors instead of destroying:

```
lifecycle { prevent_destroy = true }
```

- Review the plan carefully for -/+ symbols before every apply to production.
- In CI/CD, use terraform show -json tfplan.binary | jq to programmatically detect replacement plans and alert reviewers:

```
terraform show -json tfplan.bin | \
  jq '[.resource_changes[] | select(.change.actions | contains(["delete"]))]'
```

- Use create_before_destroy to minimise downtime when replacement is unavoidable.

**Real-World Scenario — Accidental Production DB Recreation**

An engineer updated the db_instance_identifier from 'prod-db' to 'prod-database' (for naming consistency). Terraform showed -/+ destroy/create. The engineer didn't read the plan carefully and ran apply. The production database was destroyed. Even with a recent snapshot, recovery took 4 hours including data validation.

The fix: The db_instance_identifier should never be changed once in production. Use prevent_destroy and code review checklists that specifically flag identifier changes on stateful resources.

> *Tip: Treat any -/+ (replacement) plan for a stateful resource (database, cache, Kubernetes cluster) as a P1 incident risk requiring team lead approval before proceeding. Automate this check in your pipeline.*

---

## Q21. What actually happens if two CI/CD pipelines run terraform apply at the same time? How does locking work — and when does it still fail?

**Answer**

Concurrent Terraform applies to the same state are one of the most dangerous situations in infrastructure management. State locking is the primary safeguard — but it has failure modes that every engineer must understand.

**What Happens Without Locking**

Two pipelines simultaneously read the state, each plans changes based on the same state snapshot, each writes their changes, and the second write overwrites the first. The resulting state is corrupted — containing resources from both plans that are now inconsistent.

**State Locking Mechanism — AWS**

With the S3 backend + DynamoDB locking:

- Pipeline A runs terraform plan — it reads the state and writes a lock record to DynamoDB with a unique lock ID and timestamp.
- Pipeline B runs terraform plan simultaneously — it tries to write a DynamoDB lock record but finds one already exists. It fails with:

```
Error: Error acquiring the state lock
Lock Info:
  ID:        e7e2f0ac-a3da-9f28-7bca-6b1db1e6b1ca
  Path:      prod/terraform.tfstate
  Operation: OperationTypePlan
  Created:   2024-01-15 09:32:00.000 UTC
```

```
  Who:       pipeline-run-1234 on build-agent-01
```

- Pipeline A completes, writes the state, and releases the lock (deletes the DynamoDB record).
- Pipeline B can now acquire the lock and proceed.

**State Locking — Azure**

The azurerm backend uses Azure Blob Storage lease instead of DynamoDB. Terraform acquires an exclusive lease on the state blob. A concurrent process attempting to acquire the lease receives an error immediately.

**When Locking Still Fails**

- Crash without unlock — If Terraform process is killed (OOM kill, network disconnection, pipeline timeout) mid-apply, the lock record remains in DynamoDB or the blob lease expires after 15 seconds. However, if the Terraform process held a DynamoDB lock but crashed before releasing it, the lock remains indefinitely. The team must manually force-unlock:

```
terraform force-unlock <LOCK_ID>
```

- Race condition in plan vs apply — If Pipeline A completes plan (releasing the plan lock) and Pipeline B acquires the apply lock before Pipeline A's apply lock is acquired, both could be in the apply phase simultaneously with different plans.
- Backend unavailability — If DynamoDB or Blob Storage is temporarily unavailable, Terraform may skip the lock or fail entirely — depending on the error handling.
- Custom backends without locking — The local and HTTP backends do not implement locking. Multiple concurrent applies are not prevented.

**Prevention Beyond Locking**

- Serialise applies in CI/CD — Use pipeline concurrency controls to ensure only one pipeline applies per state at a time. In GitHub Actions:

```
concurrency:
  group: terraform-apply-prod
  cancel-in-progress: false   # Queue, don't cancel
```

- Terraform Cloud/Enterprise — Built-in run queue. Applies are automatically serialised per workspace. No concurrent apply risk.
- Single pipeline per environment — Only one pipeline is configured to apply to production. No way for two pipelines to target the same state simultaneously.

> *Tip: Force-unlock should require team lead approval in production. A lock left by a crashed pipeline is different from a lock held by an active apply — unlocking an active apply causes state corruption. Verify no apply is running before force-unlocking.*

---

## Q22. Why is depends_on a last-resort solution? How does Terraform really build and evaluate the dependency graph?

**Answer**

depends_on is a powerful but blunt instrument. Understanding why it should be avoided reveals how Terraform's dependency engine actually works — and how to write code that makes it work correctly by design.

**How Terraform Builds Dependencies Without depends_on**

Terraform automatically tracks every reference between resources. When you write resource_b.attribute = resource_a.output, Terraform records: 'resource_b depends on resource_a'. This is an implicit dependency — derived from the data flow, not declared separately.

```
resource "aws_iam_role" "lambda_role" {
  name = "lambda-execution-role"
}

resource "aws_iam_role_policy" "lambda_policy" {
  role   = aws_iam_role.lambda_role.name   # ← implicit dependency
  policy = data.aws_iam_policy_document.lambda.json
}

resource "aws_lambda_function" "my_fn" {
  role = aws_iam_role.lambda_role.arn       # ← implicit dependency
}
```

Terraform's graph: iam_role → iam_role_policy, iam_role → lambda_function. Lambda and policy are both created after the role. No depends_on needed.

**When depends_on Is Required**

depends_on is needed only when a dependency EXISTS but CANNOT be expressed through attribute references:

```
# The Lambda needs the role policy attached before it executes correctly
# But the Lambda resource does not reference the policy ARN in any attribute
resource "aws_lambda_function" "my_fn" {
  role       = aws_iam_role.lambda_role.arn
  depends_on = [aws_iam_role_policy.lambda_policy]
  # Without this: Lambda might start executing before policy is attached
}
```

**Why depends_on Is a Last Resort**

- It breaks plan optimisation — When depends_on is present, Terraform must fully resolve the dependency before planning dependent resources. This eliminates parallelism for that chain.
- It hides the actual coupling — A future engineer sees depends_on but doesn't understand why. The attribute-reference version is self-documenting.
- It's too broad — depends_on creates a dependency on the ENTIRE resource, not just the attribute you need. The graph is coarser than necessary.
- It cascades during destroy — Terraform reverses depends_on during destroy too, potentially forcing unnecessary destroy ordering.

**The Real Solution — Express Dependencies Through Data**

If you find yourself reaching for depends_on, first ask: can I restructure this so that the dependency is expressed through an attribute reference?

```
# Instead of:
resource "aws_lambda_function" "fn" {
  role       = aws_iam_role.lambda_role.arn
  depends_on = [aws_iam_role_policy.lambda_policy]
}

# Better: if possible, reference the policy in a meaningful attribute
# Or use a module that bundles role + policy so the dependency is internal
```

**Module-Level depends_on**

depends_on can be applied at the module level — the entire module and all its resources depend on the referenced resource. Use this with extreme caution; it serialises all module resources:

```
module "application" {
  source     = "./modules/app"
  depends_on = [module.networking]   # All app resources wait for all networking resources
}
```

> *Tip: Before adding depends_on, spend 10 minutes trying to restructure the code so the dependency is naturally expressed through an attribute reference. This produces better, more maintainable code and allows Terraform to optimise the graph.*

---

## Q23. How can lifecycle { ignore_changes } silently break production? Explain a real failure scenario.

**Answer**

ignore_changes is one of Terraform's most dangerous features because it permanently disables drift detection for specific attributes — turning off Terraform's safety net without any visual indication in plan output.

**How ignore_changes Works**

```
resource "aws_security_group" "app" {
  name = "app-sg"
  lifecycle {
    ignore_changes = [ingress, tags]
  }
}
```

After this is applied, Terraform will NEVER show a diff for ingress rules or tags on this security group, regardless of what changes in the real infrastructure or the code. It effectively blinds Terraform to those attributes.

**Real Failure Scenario — Security Group Rule Bypass**

A team managing a fintech application added this lifecycle block 18 months ago because an external firewall tool was managing the security group rules:

```
resource "aws_security_group" "payment_processor" {
  lifecycle { ignore_changes = [ingress] }
}
```

Timeline of what happened:

- Month 1: Legitimate. The firewall tool manages ingress rules correctly.

- Month 6: The external firewall tool was replaced with a new system. Someone updated the new system's config but forgot to check the security group.
- Month 8: The old firewall tool had added an overly broad rule: TCP 0-65535 from 0.0.0.0/0. This rule persisted after the tool was replaced.
- Month 14: Security audit discovers the open security group rule.
- Root cause: ignore_changes on ingress meant Terraform never showed the broad rule in plan output. Engineers reviewing plans for 8 months saw no indication that anything was wrong.
- Impact: Any internet host could reach the payment processor on any port for 8 months. PCI-DSS violation. Regulatory notification required.

**Other Silent Failure Scenarios**

- ignore_changes = [desired_capacity] on an ASG — If the autoscaler sets capacity to 0 during an incident and the issue is never investigated, the service runs at 0 capacity indefinitely with no Terraform alarm.
- ignore_changes = [instance_type] on an EC2 — An engineer manually downgrades instance type to cut costs during a crisis. The change is never formalised. Terraform never flags it. The instance runs on the wrong type forever.
- ignore_changes = [engine_version] on RDS — Database engine is manually upgraded. Terraform never detects the version mismatch. On next apply, Terraform may use incorrect engine-version-specific features.

**Safe Use of ignore_changes**

- Document exactly WHY the attribute is ignored with a comment:

```
lifecycle {
  ignore_changes = [
    desired_capacity,  # Managed by Karpenter autoscaler — do not reset
    tags["LastUpdated"] # Updated by AWS Config — do not overwrite
  ]
}
```

- Scope narrowly — Ignore specific attributes, not entire blocks. Prefer ignore_changes = [tags["AutoScalingGroupName"]] over ignore_changes = [tags].
- Review regularly — Audit ignore_changes entries quarterly. Remove them when the external tool is decommissioned.
- Use external validation — If an attribute is ignored by Terraform, add monitoring or AWS Config rules to detect unexpected changes to it through other means.

> *Warning: Never use ignore_changes = all. This makes the resource completely unmanaged by Terraform — it will never show changes regardless of what happens to it. If you need this, consider whether Terraform should manage the resource at all.*

---

## Q24. Why does upgrading a provider sometimes recreate resources even when the configuration hasn't changed?

**Answer**

Provider upgrades are one of the most surprising causes of resource recreation. The root cause is that the provider — not your Terraform code — determines how attributes are read, computed, and compared. A provider change can alter this calculation, causing Terraform to see a difference that isn't real.

**Root Causes of Provider-Triggered Recreation**

**Cause 1 — ForceNew Attribute Added to an Existing Attribute**

The provider team changes an attribute from 'updateable' to 'forces new resource'. This is technically a breaking change but is sometimes introduced in minor version upgrades. Your configuration hasn't changed, but upgrading the provider now means that attribute requires replacement:

```
# aws provider 4.x: engine_version is updateable
# aws provider 5.x: engine_version forces replacement on certain changes
# After upgrade: terraform plan shows -/+ for a resource you never changed
```

**Cause 2 — Attribute Read/Normalisation Changed**

The provider changes how it reads or formats an attribute from the API. The real value is the same, but the provider now formats it differently in the state — causing a perpetual diff:

```
# Before upgrade: policy stored as '{"Version":"2012-10-17",...}'
# After upgrade: provider normalises to '{"Version": "2012-10-17", ...}' (whitespace)
# Terraform sees a diff → plans update → update is no-op → diff remains next plan
```

**Cause 3 — Default Value Changed**

The provider adds a new attribute with a default value in a new version. If your resource was created without this attribute (using the old default), the provider now reads the actual cloud value — which may differ from the new default:

```
# New provider adds: enable_dns_64 defaulting to true
```

```
# Your VPC was created with enable_dns_64 = false (old default was false)
# Provider upgrade: plan shows enable_dns_64 = false → true (new default)
# Applying changes the real resource — possibly causing a recreation
```

**Cause 4 — Schema Validation Tightened**

The provider validates attribute values more strictly. A value that was previously accepted is now rejected or normalised to a canonical form. This causes a mismatch between the stored state value and the normalised form.

**How to Safely Upgrade Providers**

- Step 1: Pin versions and upgrade one version at a time:

```
required_providers {
  aws = { version = "~> 5.10" }   # Pin to specific minor version
}
```

- Step 2: Read the provider CHANGELOG before upgrading. Look for: ForceNew changes, attribute normalisation changes, new required attributes.
- Step 3: Test the upgrade in dev first:

```
terraform init -upgrade
terraform plan > upgrade-plan.txt
# Review carefully for -/+ replacements before applying anywhere
```

- Step 4: If you see unexpected replacements, use lifecycle { ignore_changes } temporarily until you understand the cause, or pin to the previous provider version until the issue is resolved.

**Finding Which Provider Change Caused It**

```
# Check provider versions
terraform version
cat .terraform.lock.hcl

# Enable debug logging to see what values the provider is returning
TF_LOG=DEBUG terraform plan 2>&1 | grep -i 'update\|replace\|forcenew'
```

*Tip: Lock your .terraform.lock.hcl file into Git. This ensures all team members and CI/CD pipelines use exactly the same provider version. Only update the lock file deliberately with terraform init -upgrade, then commit the updated lock file after reviewing the plan.*

---

## Q25. What's the real difference between state drift and configuration drift? How do you detect and fix each safely?

**Answer**

These two terms are often confused but describe fundamentally different problems with different detection methods and fixes.

**State Drift — Infrastructure Diverged from State**

State drift means the REAL infrastructure changed outside of Terraform, and the state file no longer accurately records what exists:

```
What Terraform state says:    security group has 2 inbound rules
What actually exists in AWS:  security group has 3 inbound rules (one added manually)
```

Detection: terraform plan -refresh-only — Terraform calls the API, compares the real state to the stored state, and shows the difference.

```
terraform plan -refresh-only
# Output: ~ aws_security_group.app: ingress rule added
```

Fix Options:

- Accept the drift — If the manual change is valid, update the Terraform code to include it and run terraform apply. The state is updated and the code reflects reality.
- Revert the drift — If the manual change is invalid, run terraform apply (without -refresh-only). Terraform reverts the real infrastructure to match the desired state.
- terraform apply -refresh-only — Updates the state to match reality but does NOT change the infrastructure. Useful for acknowledging drift without reverting it.

**Configuration Drift — Code Diverged from State**

Configuration drift means the TERRAFORM CODE changed but terraform apply has not been run yet. The state and real infrastructure are still aligned, but the code describes something different:

```
What Terraform code says:     VPC with CIDR 10.0.0.0/8 (someone changed the code)
What Terraform state says:    VPC with CIDR 10.0.0.0/16 (not yet applied)
What actually exists in AWS:  VPC with CIDR 10.0.0.0/16
```

Detection: terraform plan (standard plan without -refresh-only) — Shows the difference between desired state (code) and current state:

```
terraform plan
# Output: ~ aws_vpc.main: cidr_block = '10.0.0.0/16' → '10.0.0.0/8'
```

Fix: terraform apply — Makes the real infrastructure match the desired code. Or revert the code change if it was a mistake.

**Comparison**

| Aspect | State Drift | Configuration Drift |
|---|---|---|
| Cause | Manual change to real infra | Code changed but not applied |
| State accurate? | No — state is out of date | Yes — state matches reality |
| Code accurate? | Yes — code is the source of truth | No — code diverged from reality |
| Detection command | terraform plan -refresh-only | terraform plan |
| Fix | Update code or revert infra | Apply code changes or revert code |
| Urgency | Often urgent — untracked security change | Normal — pending change |

**Detecting Both Simultaneously**

```
# Full plan: detects BOTH state drift and configuration drift
terraform plan
# If plan shows changes when you didn't change code → state drift
# If plan shows changes after code changes → configuration drift
# If plan shows changes after both → both types simultaneously
```

> *Tip: Run terraform plan -refresh-only regularly as a sanity check, even when you haven't changed code. This reveals state drift early — before it causes an unexpected change when someone runs a legitimate apply.*

---

## Q26. Why does terraform import almost always create unexpected diffs? What does Terraform never tell you here?

**Answer**

terraform import is deceptively simple — it seems to just add a resource to state. But it consistently produces unexpected plan diffs because of what Terraform does NOT do during import.

**What terraform import Actually Does**

terraform import ONLY does one thing: it calls the provider's Read function to get the resource's current attributes and writes them to the state file. It does NOT:

- Read your Terraform configuration (.tf files)
- Validate that your config matches the imported resource
- Generate Terraform configuration for you
- Detect attribute differences between config and reality

**Why Diffs Appear After Import**

After running terraform import, when you run terraform plan, Terraform compares your written configuration to the imported state. Almost always, the configuration you wrote doesn't exactly match how AWS stored the resource:

```
# You wrote:
resource "aws_security_group" "app" {
  name        = "app-sg"
  description = "Application security group"
  vpc_id      = "vpc-12345"
}

# After import, plan shows:
~ aws_security_group.app
  + tags_all = {}            # AWS adds account-level default tags
  ~ revoke_rules_on_delete = null → false  # Provider default differs
  + ingress { ... }          # Existing rules not in your config
  + egress  { ... }          # Existing egress rules
```

**Common Unexpected Diffs After Import**

| Attribute | Why It Appears in Plan |
|---|---|
| tags_all | AWS adds default tags from account tag policies |

| | |
|---|---|
| ingress/egress rules | Existing rules not written in your config |
| revoke_rules_on_delete | Provider default differs from actual value |
| policy | AWS normalises JSON differently than you wrote it |
| arn | Computed after creation — you didn't write it |
| account_id in policies | AWS expands wildcards to explicit account IDs |

**The Terraform import Workflow — Done Correctly**

- Step 1: Import the resource:

```
terraform import aws_security_group.app sg-0a1b2c3d
```

- Step 2: Run terraform show to see the actual imported state:

```
terraform show
```

- Step 3: Update your .tf configuration to EXACTLY match the imported state. This is the tedious part — every attribute in the state that you didn't write must either be added to config or handled with ignore_changes.
- Step 4: Run terraform plan repeatedly, updating config until plan shows zero changes.
- Step 5: Only after plan shows no changes is the import complete.

**Terraform 1.5+ — config-generate Flag**

Terraform 1.5+ introduced -generate-config-out to automatically generate configuration from imported state:

```
terraform plan -generate-config-out=imported.tf
```

This generates a .tf file with all attributes from the imported resource. Review and clean it up, but it eliminates most of the manual config-writing work.

**Terraformer — Bulk Import**

For importing many resources, use Terraformer to auto-generate both the state imports and the Terraform configuration:

```
terraformer import aws --resources=vpc,subnet,security_group --regions=us-east-1
```

> *Tip: After every terraform import, your definition of 'done' is terraform plan showing zero changes. If there are any diffs remaining, the import is not complete — the config doesn't accurately describe the imported resource.*

---

## Q27. How does Terraform deal with eventual consistency in cloud APIs? Why does this matter under load and automation?

**Answer**

Eventual consistency means that after a write operation, the cloud API may return stale data for a period before all systems converge on the new value. Terraform deals with this through built-in retry logic — but it can fail under certain conditions.

**What Is Eventual Consistency in AWS/Azure**

Most AWS services use eventually consistent data stores. When you create an IAM role, it may take 5-10 seconds before all AWS regions and services can see and use that role. During this window, operations that depend on the role may fail:

```
# Terraform creates an IAM role
aws iam create-role --role-name my-role   # Returns immediately

# Terraform immediately tries to create a Lambda using the role
aws lambda create-function --role arn:aws:iam::123:role/my-role
# Error: The role defined for the function cannot be assumed by Lambda
# Because IAM role isn't visible to Lambda service yet
```

**How Terraform Handles This**

AWS provider includes built-in waiters — after creating a resource, it polls the API until the resource is in the expected state before proceeding. For IAM roles, the provider waits until the role is usable:

```
# Provider waiter: polls every 5 seconds up to 60 seconds
# until IAM role is assumable by the target service
```

Most providers implement waiters for common eventual consistency scenarios. The provider's Read function may also retry on certain error codes (ThrottlingException, ResourceNotFoundException for newly created resources).

**When It Fails — Under Load and Automation**

- Timeout too short — If the eventual consistency period is longer than the provider's waiter timeout (uncommon but happens during AWS service disruptions), the apply fails.

- Race conditions in parallel applies — When multiple resources are created in parallel (Terraform's default 10 concurrent operations), a resource B may start before the waiter for resource A has completed. The dependency graph eliminates most of this, but misused depends_on or missing references can expose gaps.
- Automation at high frequency — CI/CD pipelines that create and destroy environments rapidly (ephemeral testing environments) are most vulnerable. Creating a VPC, waiting 200ms, then creating a subnet may fail if VPC propagation is slower than usual.
- IAM propagation — The most common failure. IAM changes propagate globally through a distributed system. In us-east-1, the role exists immediately. In ap-southeast-1, it may take 10-15 seconds longer.

**Mitigation Strategies**

- Use provider version with better waiters — Newer provider versions often improve waiter logic based on field reports.
- Add explicit time_sleep resource for known eventual consistency gaps:

```
resource "time_sleep" "wait_iam_propagation" {
  depends_on      = [aws_iam_role_policy.lambda_policy]
  create_duration = "15s"
}

resource "aws_lambda_function" "fn" {
  depends_on = [time_sleep.wait_iam_propagation]
  role       = aws_iam_role.lambda_role.arn
}
```

- Use retries in Terraform provisioners — If using provisioners (generally avoid), add retry logic.
- Monitor and alert — If applies fail with 'resource not found' errors, implement a retry step in the CI/CD pipeline before declaring failure.

> *Tip: The time_sleep resource is a legitimate workaround for well-known eventual consistency issues (IAM propagation is the most common). Document the reason in a comment — future engineers need to know why the sleep exists.*

---

## Q28. What breaks when you rename a resource without using moved blocks? And why can the damage be permanent.

**Answer**

Renaming a resource in Terraform without using a moved block is one of the most dangerous operations in Terraform. It looks trivial in the code but can cause immediate deletion of production infrastructure.

**What Renaming Means to Terraform**

Terraform identifies resources by their address: resource_type.resource_name (e.g., aws_db_instance.prod). When you rename prod to production in your .tf file, Terraform does NOT recognise this as a rename. It sees:

- aws_db_instance.prod — DELETED (was in state, now gone from code)
- aws_db_instance.production — CREATED (in code, not in state)

```
# You changed:
resource "aws_db_instance" "prod" { ... }
# To:
resource "aws_db_instance" "production" { ... }

# terraform plan shows:
- aws_db_instance.prod will be destroyed
+ aws_db_instance.production will be created

# If prevent_destroy is not set: Terraform deletes your production database
# and creates a brand new empty one. Your data is gone.
```

**Why the Damage Can Be Permanent**

- RDS databases with skip_final_snapshot = true — Deleted without a snapshot. Data permanently lost.
- S3 buckets with force_destroy = true — Bucket and all contents deleted. Irreversible.
- KMS keys — Deleted (with 7-30 day pending deletion by default, but scheduled deletion begins). All data encrypted with the key becomes permanently inaccessible if not cancelled quickly.
- EKS clusters — Deleted. All workloads, PVs, and configuration lost. Recovery requires full cluster recreation and workload redeployment.

**The Correct Way — moved Blocks**

Terraform 1.1+ moved blocks tell Terraform that a resource moved from one address to another. The state is updated, no recreation happens:

```
# Add to your .tf files alongside the renamed resource
```

```
moved {
  from = aws_db_instance.prod
  to   = aws_db_instance.production
}

# terraform plan now shows:
# aws_db_instance.prod has moved to aws_db_instance.production
# No changes. Your infrastructure is safe.
```

After the moved block is applied, you can remove it in a subsequent PR (once all environments have been updated).

**terraform state mv — Pre-1.1 Alternative**

```
# Manually update state without moved block
terraform state mv aws_db_instance.prod aws_db_instance.production
# Then rename in code
# Plan should show no changes
```

**Module Moves**

moved blocks also work for module addresses, and for resources being moved into or out of modules:

```
moved {
  from = aws_subnet.private
  to   = module.networking.aws_subnet.private
}
```

**Prevention**

- Always use prevent_destroy on stateful resources — Prevents accidental deletion even without moved blocks (Terraform errors instead of destroying).
- Always run terraform plan and read it carefully before applying any rename. A -/+ on a stateful resource is a red flag.
- Code review checklist — Any rename of a resource block must include a moved block. Reviewers should reject renames without moved blocks for stateful resources.

> *Warning: terraform state mv manipulates the state file directly. Always back up the state before using it. An incorrect state mv can corrupt the state in ways that require manual recovery.*

---

## Q29. Why are secrets still risky in Terraform variables — even with remote state? What do most teams miss?

**Answer**

Even with best practices like sensitive = true and encrypted remote state, Terraform variables holding secrets have multiple exposure vectors that most teams overlook.

**What sensitive = true Actually Provides**

- Redacts the value from terraform plan and apply console output.
- Prevents the value from appearing in Terraform Cloud run logs.
- Marks outputs as sensitive so they're not shown by default.

**What sensitive = true Does NOT Provide**

- It does NOT prevent the value from being stored in the state file — sensitive variable values are stored in plaintext in terraform.tfstate:

```
# In terraform.tfstate — plaintext regardless of sensitive=true
"resources": [{ "instances": [{ "attributes": {
  "password": "s3cr3tP@ssword"
}}]}]
```

- It does NOT encrypt the value in Terraform Cloud state (the state itself must be encrypted).
- It does NOT prevent the value from appearing in debug output (TF_LOG=DEBUG).

**The Exposure Vectors Most Teams Miss**

- 1. Remote state readable by all pipeline agents — The S3 bucket or Blob Storage state file can be read by anyone with GetObject permission. If any pipeline agent can read the state, it can extract all sensitive values with:

```
terraform state pull | jq '.resources[].instances[].attributes.password'
```

- 2. Plan files contain secrets — terraform plan -out=plan.bin includes all attribute values, including sensitive ones. The plan file is often stored as a CI/CD artifact or passed between pipeline stages:

```
terraform show -json plan.bin | jq '.. | .password? // empty'
# Outputs the password from the plan file — in plaintext
```

- 3. Logs and error messages — Despite masking in Terraform output, error messages from AWS/Azure APIs sometimes echo the value back. If TF_LOG is enabled, all API request/response bodies are logged including sensitive values.

- 4. State access history — S3 GetObject API calls to the state file are logged in CloudTrail. Anyone reviewing CloudTrail can see which pipeline agents accessed the state and when — potentially revealing that a secret was in the state.
- 5. Terraform Cloud state inspection — In Terraform Cloud, workspace members with State access can view the raw state file — including sensitive values.

**What Teams Miss — Plan File Security**

The plan file is particularly dangerous. In CI/CD pipelines, the plan file is saved as an artifact to pass between pipeline stages:

```
# Stage 1: Save plan
terraform plan -out=prod.plan
# prod.plan is uploaded as a CI/CD artifact — often publicly accessible

# Stage 2: Apply saved plan
terraform apply prod.plan
```

If the artifact storage is not properly secured, anyone who can access the CI/CD artifacts can extract all secrets from the plan file.

**The Correct Approach — Don't Put Secrets in Variables**

The root fix is not to pass secrets as Terraform variables at all. Instead, retrieve them from a secret manager using data sources (see Q2). The secret never appears in variables, plan files, or the state in a meaningful way:

```
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "prod/db/credentials"
}
# The ARN is in state; the secret VALUE is fetched at apply time
```

*Tip: Audit your state files and plan artifacts for sensitive values. Use jq to search: terraform state pull | jq '.. | strings | select(test("password|secret|key|token"; "i"))'. If you find secrets in state, migrate to data source retrieval and rotate the exposed credentials.*

---

## Q30. Why is switching from count to for_each dangerous? What happens internally to the state?

**Answer**

Switching from count to for_each in existing Terraform code is dangerous because it changes how Terraform identifies resources in the state file. Without moved blocks, Terraform sees this as deleting all count-indexed resources and creating new for_each-keyed resources — destroying and recreating production infrastructure.

**Internal State Representation**

With count, each instance is stored in state using numeric index:

```
# State with count
"aws_subnet.private": {
  "instances": {
    "0": { "cidr_block": "10.0.1.0/24", "az": "us-east-1a" },
    "1": { "cidr_block": "10.0.2.0/24", "az": "us-east-1b" }
  }
}
```

With for_each, each instance is stored using the map key:

```
# State with for_each
"aws_subnet.private": {
  "instances": {
    "subnet-a": { "cidr_block": "10.0.1.0/24", "az": "us-east-1a" },
    "subnet-b": { "cidr_block": "10.0.2.0/24", "az": "us-east-1b" }
  }
}
```

**What Terraform Sees Without moved Blocks**

```
# Before: using count
resource "aws_subnet" "private" { count = 2 ... }
# State: aws_subnet.private[0], aws_subnet.private[1]

# After: switched to for_each
resource "aws_subnet" "private" { for_each = {"a":{},"b":{}} ... }
# Code: aws_subnet.private["a"], aws_subnet.private["b"]

# terraform plan shows:
- aws_subnet.private[0]    # Destroy (was in state, not in new code)
- aws_subnet.private[1]    # Destroy
+ aws_subnet.private["a"] # Create (new key)
```

```
+ aws_subnet.private["b"] # Create

# Result: Terraform destroys both subnets and creates two new ones
# All EC2 instances, EKS nodes, and RDS in those subnets lose networking
```

**The Safe Migration Path — moved Blocks**

Add a moved block for each count index → for_each key mapping. This updates the state keys without destroying and recreating:

```
moved {
  from = aws_subnet.private[0]
  to   = aws_subnet.private["subnet-a"]
}

moved {
  from = aws_subnet.private[1]
  to   = aws_subnet.private["subnet-b"]
}

# terraform plan now shows:
# aws_subnet.private[0] has moved to aws_subnet.private["subnet-a"]
# aws_subnet.private[1] has moved to aws_subnet.private["subnet-b"]
# Plan: No changes.
```

**Alternative — terraform state mv**

```
# Before changing code, update state directly
terraform state mv 'aws_subnet.private[0]' 'aws_subnet.private["subnet-a"]'
terraform state mv 'aws_subnet.private[1]' 'aws_subnet.private["subnet-b"]'
# Then change code from count to for_each
# Run plan — should show no changes
```

> *Warning: For modules using count, the moved block must reference module addresses: moved { from = module.service[0]; to = module.service["auth"] }. Test in dev and verify plan shows only moved statements — no destroys — before applying to production.*

---

## Q31. How do implicit dependencies fail during refactoring? And why doesn't Terraform warn you?

**Answer**

Implicit dependencies — the ones Terraform derives from attribute references — are the safest kind of dependency. But during refactoring, they can disappear silently, causing ordering issues that Terraform never detects or warns about.

**How Implicit Dependencies Are Created**

Terraform tracks every reference. When resource B uses an attribute of resource A, the graph records that B depends on A. This is implicit — derived from the code structure, not declared:

```
resource "aws_iam_role" "lambda_role" { name = "my-role" }

resource "aws_lambda_function" "fn" {
  role = aws_iam_role.lambda_role.arn  # ← implicit dep on lambda_role
}
```

**How Refactoring Breaks Implicit Dependencies**

During refactoring, you extract resources into modules. Inside the module, the same resources exist. Outside the module, the reference is now to a module output — which looks the same to the caller but the internal dependency graph has changed:

```
# Before refactoring:
resource "aws_iam_role" "lambda_role" { ... }
resource "aws_iam_role_policy" "policy" {
  role = aws_iam_role.lambda_role.name  # implicit dep on role
}
resource "aws_lambda_function" "fn" {
  role = aws_iam_role.lambda_role.arn   # implicit dep on role
}

# After refactoring — IAM role moved into module:
module "iam" {
  source = "./modules/iam"
}
```

```
resource "aws_lambda_function" "fn" {
  role = module.iam.role_arn   # still has implicit dep on module.iam
}

# BUT inside the module:
# The policy attachment is now separate from the role creation
# The lambda_function depends on module.iam (for the ARN)
# But NOT specifically on the policy attachment inside the module
# Terraform may create the Lambda before the policy is fully propagated
```

**Why Terraform Does Not Warn You**

From Terraform's graph perspective, the Lambda depends on the module output (role ARN). The module output is computed when the role is created. Terraform has no way to know that the POLICY inside the module also needs to be applied before the Lambda is usable. The dependency on the module is satisfied when the role exists — not when the policy is attached.

This is a semantic dependency (the Lambda needs the policy to work correctly) versus a structural dependency (the Lambda needs the role ARN to be created). Terraform only understands structural dependencies expressed through attribute references.

**The Fix — Explicit Dependency or Module Output**

- Option 1: Use depends_on to express the semantic dependency:

```
resource "aws_lambda_function" "fn" {
  role       = module.iam.role_arn
  depends_on = [module.iam]   # Wait for ALL module resources, including policy
}
```

- Option 2: Expose a signal output from the module that includes the policy:

```
# Inside module: outputs.tf
output "role_arn_after_policy" {
  # Include policy in the output computation to create the dependency
  value = "${aws_iam_role.main.arn}${aws_iam_role_policy.main.id ==
aws_iam_role_policy.main.id ? "" : ""}"
}
```

**How to Detect Missing Dependencies**

- Test in isolation — Apply the infrastructure from scratch in a clean environment. Dependency issues often appear on first-time creation when resources race.
- Run terraform apply with -parallelism=1 — Forces sequential creation. Then increase parallelism and observe if failures occur.
- Check apply logs for race condition errors — 'InvalidParameterValueException: The role defined for the function cannot be assumed by Lambda' is a classic IAM propagation race.

> *Tip: After any significant refactoring (extracting modules, renaming resources), apply to a fresh environment from scratch. Race conditions and missing dependencies only appear during creation — an incremental apply may succeed simply because all resources already exist.*

---

## Q32. Why does terraform destroy sometimes fail halfway? How do you recover cleanly without rebuilding everything?

**Answer**

terraform destroy is a bulk operation that follows the dependency graph in reverse. When it fails midway, you're left with a partially destroyed environment — some resources destroyed, some still existing, and a state file that is partially updated.

**Why Destroy Fails Midway**

- Resource protection — A resource in the dependency chain has deletion protection enabled (RDS deletion_protection = true, S3 force_destroy = false with objects still in bucket).
- API errors — Transient AWS/Azure API errors (ThrottlingException, timeout). The provider retries but eventually gives up.
- Dependencies not resolved — Trying to destroy a VPC while EC2 instances or network interfaces still exist in it. AWS returns an error.
- IAM permission missing — The Terraform pipeline role doesn't have permission to delete a specific resource type.
- Resource already deleted — Someone manually deleted a resource. Terraform's destroy call fails because the resource no longer exists (the state still references it).

**Recovery Strategy — Do NOT Rerun terraform destroy Blindly**

After a partial destroy, terraform plan shows the current partial state. Review carefully what still exists:

```
terraform plan
# Shows which resources still exist in state
# Shows which have been deleted
```

**Fix Root Cause First**

- If deletion protection is the issue, disable it in code then re-run destroy:

```
resource "aws_db_instance" "main" {
  deletion_protection = false   # Changed from true
}
terraform apply   # Apply this change first
terraform destroy # Now destroy succeeds
```

- If an S3 bucket has objects preventing deletion, empty it first:

```
aws s3 rm s3://my-bucket --recursive
terraform destroy   # Bucket is now empty, destroy succeeds
```

- If a resource was already manually deleted, remove it from state:

```
terraform state rm aws_instance.web   # Remove from state without destroying
terraform destroy   # Continue without trying to delete this resource
```

**Target-Based Destroy — Destroy Remaining Resources**

Use -target to destroy specific resources that the previous destroy missed, in the correct order:

```
# Destroy remaining resources in dependency order
terraform destroy -target=aws_instance.app   # First: app server
terraform destroy -target=aws_db_instance.main  # Then: database
terraform destroy -target=aws_vpc.main          # Finally: networking
```

**Remove from State Without Destroying (When Resource Was Manually Deleted)**

```
terraform state rm <resource_address>
```

After removing a manually deleted resource from state, terraform destroy can continue without trying to delete it again.

**The Nuclear Option — terraform state rm Everything**

For a completely broken destroy, you can remove all remaining resources from state and clean up manually (via console or AWS CLI). This abandons Terraform management of the remaining resources:

```
# Remove all resources from state
terraform state list | xargs -I{} terraform state rm {}
# State is now empty; resources still exist in cloud
# Clean up remaining resources manually
```

> *Warning: Abandoning state management should be a last resort. Resources orphaned from state are no longer tracked and can become forgotten cost centres or security risks. Always prefer fixing the destroy sequence properly.*

---

## Q33. Why does the same Terraform code generate different plans on different machines?

**Answer**

This is a common and frustrating problem in teams. The same repository produces different plans when run on different machines — plans that diverge enough to cause unexpected changes. Several mechanisms cause this.

**Cause 1 — Different Provider Versions**

If the .terraform.lock.hcl file is not committed to Git, each machine runs terraform init and may resolve a different provider version:

```
# Machine A: installed aws provider 5.10.0
# Machine B: installed aws provider 5.15.0 (newer patch available)
# Provider 5.15.0 normalises a JSON attribute differently → different plan
```

Fix: Commit .terraform.lock.hcl to Git. This pins provider versions across all machines and CI/CD agents. Only update deliberately with terraform init -upgrade.

**Cause 2 — Different Terraform Binary Versions**

```
# Machine A: Terraform 1.5.0
# Machine B: Terraform 1.7.2
# Different evaluation engines may produce different plans for edge cases
```

Fix: Pin the Terraform version using a .terraform-version file (used by tfenv) or specify the required version in the configuration:

```
terraform {
  required_version = "= 1.7.2"   # Exact version pin
}

# Use tfenv to automatically switch to the pinned version
cat .terraform-version  # Contains: 1.7.2
```

**Cause 3 — Different AWS/Azure Credentials**

Different machines use different IAM roles or service principals. If the plan reads data sources (aws_ami, aws_availability_zones, aws_caller_identity), different credentials may return different values:

```
data "aws_ami" "amazon_linux" {
  most_recent = true   # Returns different AMI in different accounts
  owners      = ["amazon"]
}
```

Machine A (dev account): returns ami-dev-latest. Machine B (prod account): returns ami-prod-latest. Different AMI IDs → different plan.

**Cause 4 — Local Variable Values**

If the plan reads local environment state (hostname, username, environment variable), it produces different values per machine:

```
# Using system information in Terraform locals — dangerous
locals {
  deployer = "${get_env("USER", "unknown")}"   # Different per machine
}
```

**Cause 5 — Non-Deterministic Functions**

```
# timestamp() — different on every machine, every run
resource "aws_ssm_parameter" "deploy_time" {
  value = timestamp()
}
```

**Cause 6 — State Staleness**

Machine A ran terraform apply 10 minutes ago, updating the state. Machine B pulls old code or has a cached local state. Plans diverge because they're working from different state versions.

Fix: Always use remote state. Never use local state in team environments. Terraform fetches the current remote state at plan time, ensuring all machines work from the same baseline.

**Creating Reproducible Plans**

| Issue | Fix |
|---|---|
| Provider version differences | Commit .terraform.lock.hcl |
| Terraform version differences | Use tfenv + .terraform-version |
| Credential-dependent data sources | Use static references where possible; document data source outputs |
| Non-deterministic functions | Avoid timestamp(), uuid() in resource arguments |
| State staleness | Always use remote state; never commit .terraform.tfstate |

> *Tip: In CI/CD, always run terraform init (which downloads from lock file) rather than assuming providers are cached. The CI/CD environment should be treated as a clean machine every time — this ensures the pipeline always uses the exact locked versions.*

---

## Q34. Terraform remote_state backend suddenly times out. What's your recovery and damage containment strategy?

**Answer**

A backend timeout during an operation means Terraform cannot read or write the state. This is a critical operational issue — any in-progress apply may leave the state in an inconsistent condition.

**Immediate Response**

- Step 1: Do not run any more Terraform commands — Additional commands against a timing-out backend can cause partial state writes or leave the lock in an inconsistent state.
- Step 2: Identify the scope — What was running when the timeout occurred? Was it a plan (read-only) or an apply (state write in progress)?

```
# Check backend availability
aws s3 ls s3://my-tfstate-bucket/prod/   # AWS
az storage blob show --account-name tfstateaccount --container-name tfstate --name
prod/terraform.tfstate   # Azure
```

**Diagnose the Root Cause**

| Backend | Common Timeout Causes |
|---|---|
| AWS S3 | S3 service disruption, IAM role expired, VPC endpoint issue |
| Azure Blob | Storage account throttling, private endpoint down, auth expired |
| Terraform Cloud | API outage, workspace locked |
| GCS | GCS quota exceeded, IAM issue |

**Check for Stale Lock**

If a pipeline was in progress during the timeout, a stale lock may exist:

```
# AWS DynamoDB — check for lock record
aws dynamodb get-item \
  --table-name terraform-state-lock \
  --key '{"LockID": {"S": "my-bucket/prod/terraform.tfstate"}}'

# If lock exists and no pipeline is running, force-unlock
terraform force-unlock <LOCK_ID_FROM_ERROR>
```

**Verify State Integrity After Backend Recovers**

```
# Once backend is accessible
terraform init    # Reconnect to backend
terraform show    # Check state is readable
terraform plan    # Verify plan matches expectations
```

If terraform plan shows unexpected changes after the backend recovers, the state may have been partially written. Compare with the previous state version (from versioning history) to determine what was partially applied.

**Damage Containment — Infrastructure May Be Partially Applied**

If terraform apply was running during the timeout, some resources may have been created/modified but not recorded in state (apply succeeded in cloud, state write failed). These are orphaned resources — they exist but Terraform doesn't know about them.

- Identify orphans — Check AWS/Azure for recently created resources (CloudTrail, Activity Log) in the timeframe of the failed apply.
- Import orphans — Use terraform import to add orphaned resources back to state before running any further applies.
- Verify with plan — After importing, run terraform plan to confirm the state accurately reflects reality.

**Backend Resilience Improvements**

- Multi-region S3 replication — State bucket replicated to another region. If primary region is down, switch backend config to replica.
- Retry logic in CI/CD — Wrap Terraform commands with retry on backend timeout:

```
for i in 1 2 3; do
  terraform apply && break
  echo 'Terraform apply failed, retrying...'
  sleep 30
done
```

- Terraform Cloud — Backend is a managed service with SLA. Consider migrating critical infrastructure to Terraform Cloud for better backend reliability.

> *Tip: After any backend timeout incident, conduct a state integrity review: run terraform plan and verify it shows only the expected changes. If unexpected changes appear, the state was partially written and needs investigation before any further applies.*

---

## Q35. How do you structure Terraform code for a multi-region, multi-account AWS setup?

**Answer**

This question overlaps with Q7 and Q4 but focuses specifically on AWS multi-account patterns. The complete answer covers account structure, code organisation, provider configuration, and state management.

**AWS Account and OU Structure (Recommended)**

```
AWS Organization
├── Management Account (billing, org-level controls only)
├── Security OU
│   ├── Security Account (GuardDuty master, Config aggregator)
```

```
│   └── Log Archive Account (centralised CloudTrail, VPC Flow Logs)
├── Infrastructure OU
│   └── Shared Services Account (ECR, Transit Gateway, DNS)
└── Workloads OU
    ├── Dev Account
    ├── Staging Account
    └── Production Account
```

**Terraform Directory Structure**

```
terraform/
├── modules/                        # Shared, versioned modules
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── global/                         # Organization-level resources
│   ├── org-scp/                   # Service Control Policies
│   └── shared-ecr/               # Cross-account ECR
└── accounts/
    ├── dev/
    │   ├── us-east-1/
    │   │   ├── networking/        # VPC, subnets — own state
    │   │   ├── compute/           # EKS — own state
    │   │   └── data/              # RDS — own state
    │   └── eu-west-1/
    ├── staging/
    └── prod/
        ├── us-east-1/
        └── eu-west-1/
```

**Provider Configuration with Role Assumption**

```
# accounts/prod/us-east-1/networking/providers.tf
provider "aws" {
  region = "us-east-1"
  assume_role {
    role_arn     = "arn:aws:iam::PROD_ACCOUNT_ID:role/TerraformRole"
    session_name = "terraform-prod-networking"
  }
}
```

**Cross-Account Cross-Region Remote State**

When the prod compute layer needs VPC IDs from the prod networking layer:

```
data "terraform_remote_state" "networking" {
  backend = "s3"
  config = {
    bucket         = "central-tfstate"
    key            = "accounts/prod/us-east-1/networking/terraform.tfstate"
    region         = "us-east-1"
    role_arn       = "arn:aws:iam::MGMT_ACCOUNT:role/TerraformStateReader"
  }
}
```

**Terragrunt for DRY Multi-Account Configuration**

Terragrunt eliminates the repetitive backend and provider configuration across 20+ account/region combinations:

```
# terragrunt.hcl at root
remote_state {
  backend = "s3"
  config = {
    bucket  = "central-tfstate"
    key     = "${path_relative_to_include()}/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}

# Account-level terragrunt.hcl
inputs = {
  account_id = "123456789012"
  account_name = "prod"
}
```

**CI/CD for Multi-Account**

GitHub Actions matrix deploys to all account/region combinations in parallel:

```
strategy:
  matrix:
    include:
    - account: dev,    region: us-east-1, role: arn:aws:iam::DEV:role/TFRole
    - account: prod,   region: us-east-1, role: arn:aws:iam::PROD:role/TFRole
    - account: prod,   region: eu-west-1, role: arn:aws:iam::PROD:role/TFRole
```

*Tip: Use AWS IAM Identity Center (SSO) for both human and CI/CD access to multiple accounts. The pipeline authenticates via OIDC to the management account and assumes cross-account roles in each target account — no long-lived credentials needed.*

---

## Q36. How do you manage secrets and config securely at scale in Kubernetes without compromising GitOps workflows?

**Answer**

This question sits at the intersection of Kubernetes, GitOps, and secret management. The core challenge: GitOps requires everything in Git, but secrets must never be in Git in plaintext. Several patterns resolve this tension.

**The Core Problem**

In a GitOps workflow (Argo CD, Flux), everything is declared in Git and the cluster reconciles to match. If you store Kubernetes Secrets in Git, those base64-encoded values are readable by anyone with repo access. Base64 is encoding, not encryption.

**Solution 1 — Sealed Secrets (Bitnami)**

Sealed Secrets encrypt Kubernetes Secret objects client-side using the cluster's public key. The encrypted SealedSecret resource is safe to commit to Git. Only the cluster (with the private key) can decrypt it:

```
# Seal a secret
kubectl create secret generic db-creds \
  --from-literal=password='s3cr3t' \
  --dry-run=client -o yaml | \
  kubeseal --controller-name=sealed-secrets \
    --format yaml > db-creds-sealed.yaml

# db-creds-sealed.yaml is safe to commit to Git
# The cluster decrypts it and creates the real Kubernetes Secret
```

**Solution 2 — External Secrets Operator (ESO)**

ESO syncs secrets FROM an external store (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) INTO Kubernetes Secrets. Only the ExternalSecret resource definition is in Git — not the secret values:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials
  data:
  - secretKey: password
    remoteRef:
      key: prod/db/credentials
      property: password
```

This YAML is 100% safe to commit. No secret values. ESO fetches the value from Secrets Manager and creates the Kubernetes Secret automatically.

**Solution 3 — Secrets Store CSI Driver**

Mounts secrets directly as files in pods from an external store, without creating Kubernetes Secrets at all:

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: prod/db/credentials
        objectType: secretsmanager
```

**Solution 4 — SOPS (Secrets OPerationS) + Age/KMS**

SOPS encrypts specific fields in YAML/JSON files using age keys or cloud KMS. The encrypted file is committed to Git. Argo CD with SOPS plugin decrypts at deploy time:

```
# Encrypt
sops --encrypt --kms arn:aws:kms:us-east-1:123:key/xxx secret.yaml > secret.enc.yaml

# Committed to Git: secret.enc.yaml (encrypted values)
# Argo CD with helm-secrets or sops plugin decrypts at apply time
```

**Choosing the Right Approach**

| Solution | Best For |
|---|---|
| Sealed Secrets | Self-contained K8s clusters, no external secret store |
| External Secrets Operator | Existing secrets in AWS/Azure, multi-cluster |
| Secrets Store CSI | No Kubernetes Secret objects needed, file-based |
| SOPS + KMS | Helm chart values with secrets, full GitOps |

**Terraform Integration**

If Terraform provisions the Kubernetes cluster, it can also write initial secrets using the Kubernetes provider — but those secret values come from AWS Secrets Manager data sources, not hardcoded values:

```
resource "kubernetes_secret" "db_creds" {
  data = {
    password = data.aws_secretsmanager_secret_version.db.secret_string
  }
}
```

> *Tip: External Secrets Operator with AWS Secrets Manager is the most operationally robust solution for production. It handles automatic rotation (when the secret rotates in Secrets Manager, ESO refreshes the K8s Secret), multi-cluster sync, and provides a clean separation between infrastructure (secrets in Secrets Manager) and application configuration (ExternalSecret in Git).*

---